Appunti di Programmazione 2

Alessio Novel

Introduzione, richiami c++, modello di memoria....2
Java....4
    Classe....4
    Package....6
    Oggetti....7
    Array....7
  Ereditarietà e polimorfismo....9
  Gerarchia di ereditarietà....10
  Costruttori....11
  Binding....11
    Regole....13
  Classi astratte....13
  Information hiding....14
    Generics....17
  Interfacce....23
  Conversioni....23
    Da stringhe a numeri....23
  Errori....23
  Collections....24
  Classi interne....28
  Uguaglianza e identità....28
    Confronto....33
  Clonazione....34
  Static....36
  Javadoc....38
Java FX....39
  Stage....40
  Eventi....47
    Lambda expressions....55
    Gestire la pressione di tasti della tastiera....56
  Posizionamento componenti....60
  Application....77

Introduzione, richiami c++, modello di memoria

Le pratiche importanti nella programmazione, chiamate “programming in the large”, sono il lavoro in team (suddivisione del lavoro), la manutenibilità del software (facilità con cui si possono cambiare le cose anni dopo) e la robustezza (cioè se cambia un programmatore)
Per fare questo si fa uso di ingegneria del software, buone tecniche di programmazione (indentazione, commenti), e un supporto da parte del linguaggio di programmazione.
Per questo si studia la programmazione ad oggetti, che aiuta a lavorare con la crescente complessità dei software.
Le istruzioni, i commenti, le strutture di controllo (iterazioni) e le definizioni/chiamate a funzioni restano invariate da C a Java.
Java obbliga la programmazione object-oriented, C++ invece lo permette
Java deriva da C quindi la sintassi è molto simile a C++, però in Java non esistono puntatori, struct e aggiunge gli oggetti (classi,ereditarietà eccetera), permette la concorrenza e introduce una serie di classi predefinite che sono diventate parte del linguaggio (ad esempio le stringhe)
Visto che molti errori nella programmazione sono dovuti ai puntatori, Java ha deciso di toglierli (in realtà solo di togliere l’aritmetica dei puntatori e “nasconderli” al programmatore) e di creare il Garbage Collection, il quale elimina i costrutti espliciti di allocazione e deallocazione e in base a diverse tecniche per gestire in automatico la deallocazione.

Un programma Java è organizzato come un insieme di **classi**.
Il programma principale è rappresentato da una funzione speciale (**main**) della classe il cui nome coincide col nome del programma.
Esiste una classe fatta apposta per le stringhe, una per leggere una stringa da GUI
In C++ stack e heap si dividono la memoria, e ogni volta che viene chiamata una funzione aumenta lo stack (e diminuisce appena la funzione termina), questo fa sì che le funzioni ricorsive possano facilmente riempire lo stack
In Java non serve l’operatore sizeof essendo che non dipende dalla macchina e sappiamo con certezza quanto occupa ogni tipo su ogni macchina
Il C++ privilegia l’efficienza piuttosto che la sicurezza, per questo non vengono segnalati errori di out bounds of array, Java invece fa la cosa opposta
Il problema delle variabili globali è che nella situazione di errore, in cui la variabile contiene qualcosa di inaspettato, bisogna controllare tutto il codice essendo che è visibile all’interno di ogni funzione (questo si chiama **side-effects**). Le variabili locali “schermano” le variabili globali e le variabili locali più esterne
Il problema del GOTO è che non si sa da dove si arriva
Fondamentale nella programmazione il principio NEED TO KNOW (esempio militari), ecco perchè è preferibile non fare uso delle variabili globali



David Parnas

Per poter cambiare il valore di una variabile passata ad una funzione devo passare per copia l'indirizzo di quella variabile.
Quando si alloca memoria nello heap bisogna stare attenti perchè se ci si dimentica la deallocazione si causa un insidioso tipo di errore: **memory leak**; e si rischia di finire la memoria con oggetti "inutili"
Il typedef aiuta a rendere il codice più leggibile

Dichiarare un vettore è in un certo senso equivalente a dichiarare un puntatore, questo perchè l'array sarà semplicemente un puntatore al primo elemento dell'array
È impossibile aumentare la lunghezza di un array perché la memoria successiva potrebbe essere allocata per qualcos'altro, l'unico modo per farlo è creare un altro array di lunghezza +1 e ricopiarci dentro quello precedente

Quando è implementato un tipo di dato astratto ad esempio, il file .h serve a nascondere l'implementazione, così facendo chi usa la nostra implementazione non vede il codice, e di conseguenza non può usarlo facendo assunzioni in base a come è fatto il programma. Così facendo se in futuro cambiamo l'implementazione (ma non l'interfaccia) dal lato utente non cambierà nulla.
Bisogna stare attenti quando si fa la copia di un oggetto puntatore perchè si rischia di fare una copia troppo sbrigativa e non è detto che sia quello che vogliamo

In una implementazione non OO abbiamo che la struttura dati è slegata dalle funzioni che la manipolano, nel senso che possiamo anche manipolarla senza usufruire delle funzioni, e non riusciamo bene a distinguere quali funzioni la utilizzano veramente e quali solo la passano come parametro per esempio.
Per iniziare a implementare una programmazione O-O quando ho un tipo di dato astratto, inizio a mettere le funzioni che la manipolano dentro alla struct del dato, così da poterla richiamare in questo modo: $pila.push();$ oppure $pila->push();$ invece di $push(pila)$, queste funzioni le chiameremo member functions (o metodi)

Dove dichiaro il codice?

- O direttamente dentro la definizione nella struct
- Oppure fuori dalla struct chiamandola $\int \; Pila::push()$

Le funzioni interne alla struct hanno un parametro implicito che è la struct stessa che l'ha chiamata, per potersi riferire ad essa bisogna usare *this* all'interno del codice della funzione, questo *this* può anche rimanere implicito perché esiste un meccanismo di risoluzione dei nomi

La funzione *crea* viene trasformata in una funzione che ha il nome della struttura ($Pila::Pila$), e nel Main quando vorrò avere una pila scriverò $Pila \;\; k=new \;\; Pila();$, la quale allocherà la memoria e in più chiamerà la funzione **costruttore**, che sarà proprio la funzione col nome della struttura
In maniera analoga la funzione *distruggi* diventerà $Pila::\sim Pila$, la quale viene chiamata **distruttore**
Quando viene fatto nel main $delete \;\; pila;$ in automatico viene chiamata la funzione $\sim Pila$
Esiste il copy constructor che viene chiamato quando si fa l'assegnazione tra oggetti
Per esempio $Pila(const \;\; Pila\& \;\; origine)\{\};$

I dati all'interno della struct vengono chiamate variabili d'istanza (o dati membro)
Il problema ora però è che un utente può accedere ancora ai dati in maniera diretta dal main tipo $pila->dato$, e per il principio di Parnas bisogna evitarlo.
L'utente deve vedere **solo** le funzioni che deve usare, quindi dati e metodi interni non devono essere visibili dal main, questo si fa usando la keyword $private$: prima dei dati e delle funzioni "interne" (questa pratica si chiama **incapsulamento** oppure **information hiding**).
Quindi se da un'istanza nel main provo ad accedere a un dato/funzione privato mi verrà segnalato un errore.
Quando definisco una *class* invece di una *struct* di default i dati e le funzioni sono privati, quindi devo rendere pubbliche ($public$:) le funzioni (i metodi) che l'utente può vedere e chiamare.

Java

Classe

In Java le classi si dichiarano così: $public \;\; class \;\; Pila$
In Java di default tutti gli attributi sono privati e tutti i metodi sono pubblici
Con tutte le dichiarazioni sotto
Il costruttore: $public \;\; Pila();$
metodi privati: $private \;\; metodo();$
In Java il nome delle classi si scrivono con la prima lettera maiuscola

In Java le costanti si definiscono con $final \;\; tipo \;\; NOME \;\; = \;\; valore;$ e per convenzione i nomi delle costanti si scrivono in maiuscolo

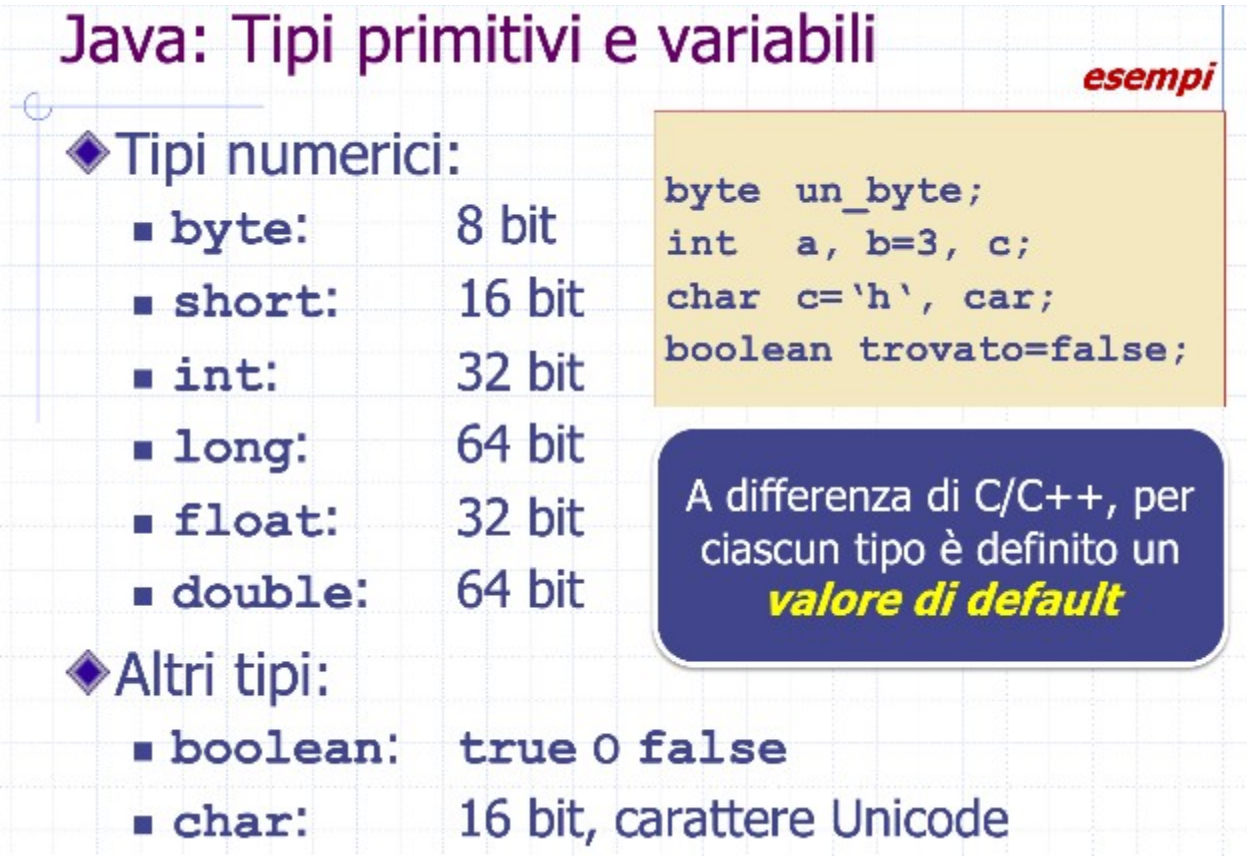
All'interno dei metodi si usa $this.metodo()$ o $this.dato$ invece di usare la freccetta per le struct nello stack, questo perché gli oggetti sono sempre allocati nello heap e sono dei riferimenti, si usa la sintassi "normale", anche qui il $this$ si può omettere

Le assert in java vengono eseguite solamente se da command line aggiungo il comando $-ea$ quando runno il programma, ad esempio $java\ -ea\ Pila$, e hanno la possibilità di aggiungere dei commenti personalizzati

In Java non serve che il programmatore pensi alla deallocazione dinamica perchè ci pensa direttamente Java, con il Garbage Collector
Il GC si attiva automaticamente quando serve memoria, e elimina intelligentemente tutti gli oggetti per i quali non esistono più riferimenti attivi
In Java si può scrivere un metodo $finalize();$, il quale viene chiamato dalla garbage collector prima di eliminare un oggetto dalla memoria.
Volendo lo si può attivare esplicitamente con $System.gc();$, ma di solito non serve e non è sicuro che venga veramente attivato (può decidere da solo che in quel momento non serva che si attivi)
Non abbiamo la sicurezza che venga effettivamente eseguito il metodo $finalize();$ perché può succedere che il programma termini prima che entri in azione il garbage collector

In realtà non è che Java non abbia i puntatori, però Java elimina l'aritmetica dei puntatori, visto che è quello il problema degli errori dei puntatori

In Java per ogni tipo è definito un valore di default che vale per ogni macchina su cui giri il codice, questo favorisce la portabilità dei programmi:



In più Java mette a disposizione tipi di dato aggiuntivi, ad esempio le String

Java introduce e migliora i booleani, ad esempio l'argomento di un istruzione *if* in Java **deve** essere un booleano, non come in C++ dove andavano bene anche interi
In C++ per esempio era comune l'errore di scrivere $if\,(i=1)$, questa operazione, assegnava il valore 1 ad i e poi controllava il valore di i che essendo diverso da zero equivaleva a *true*, in Java questo errore, sintatticamente corretto ma sbagliato, viene segnalato

In Java ogni classe può avere il suo main

Package

Il package è un insieme di classi, e contiene un insieme di classi public e private.
Tutti gli attributi e i metodi per cui non è dichiarato nessun tipo di visibilità sono visibili in tutte le classi all'interno dello stesso package.
Solo le classi public possono essere importati in altri package

Ogni compilation unit contiene classi appartenenti allo stesso package, ma contiene solo una classe public (la prima), le eventuali altre devono essere private

1 classe = 1 file

Il package $java.lang$ contiene classi di uso molto frequente (ad esempio String, Object), però non è necessario importarle prima di utilizzarle (sono importate di default)

Oggetti

Nuovi oggetti vengono creati con $nomeClasse\ \ oggetto = new\ \ nomeClasse;$ e viene chiamato il metodo costruttore della classe, il quale esegue l'allocazione della memoria e la inizializzazione.
In una classe si possono definire più costruttori (overloading)
Il compilatore inserisce un costruttore di default senza parametri (ovviamente solo nel caso in cui non ne abbiamo messo uno noi con la stessa firma) che inizializza a un valore di default tutti gli attributi primitivi e inizializza a null tutti i riferimenti.

Per riferirci ai parametri dell'oggetto si usa la sintassi $this.parametro$, nel caso in cui non ci siano altre variabili più "vicine" con quel nome si può omettere.
All'interno di un costruttore ne posso chiamare un altro, ad esempio chiamandone uno più specifico e usandone uno più basilare per poi aggiungere altri dettagli, con la notazione $this(parametri);$. Se si usa questa pratica bisogna mettere questa istruzione obbligatoriamente come prima istruzione del costruttore.

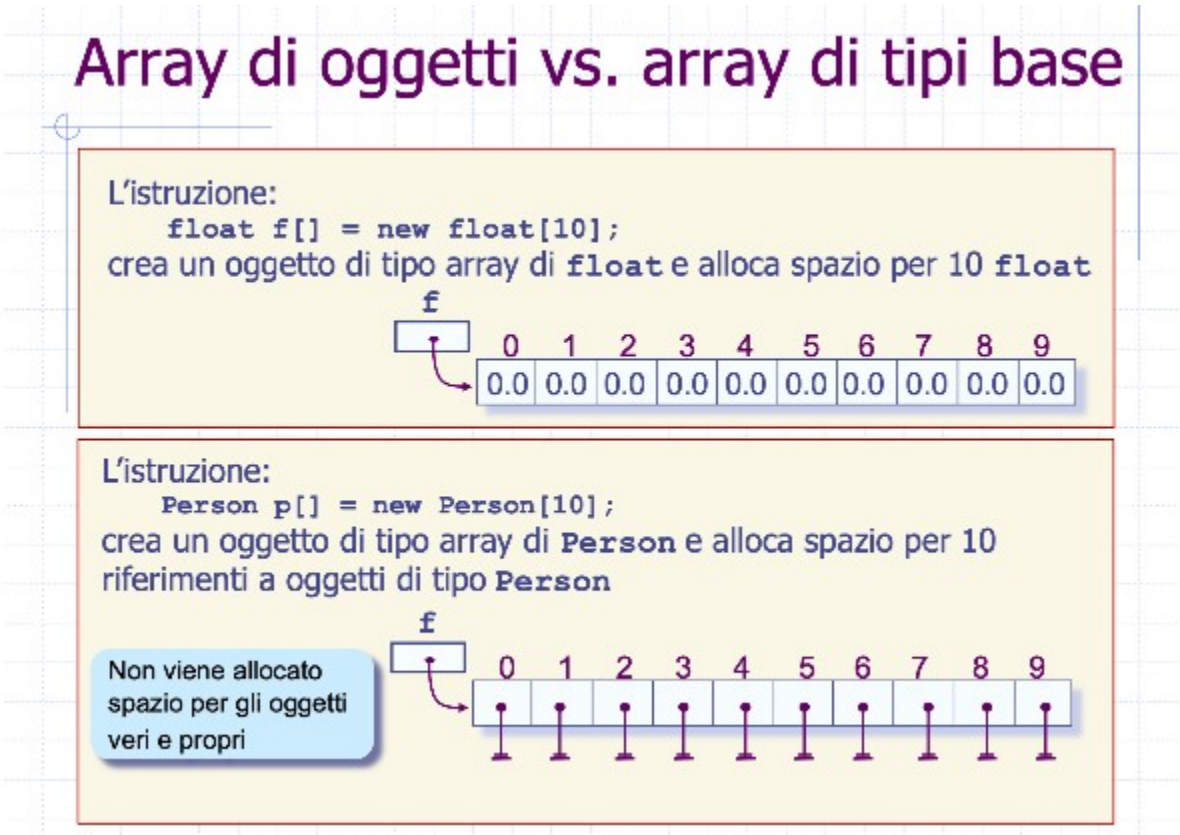
Il valore degli attributi cambia (in genere) per ogni oggetto, i metodi sono il comportamento dell'oggetto e il loro codice è lo stesso per ogni oggetto

Le istanze di una classe si chiamano **oggetti**, ogni variabile il cui tipo sia una classe contiene un riferimento ad un oggetto.
In Java infatti gli oggetti sono accessibili **solo** per riferimento (infatti vengono sempre allocati nella heap e non nello stack, in C++ invece possono anche essere dichiarati sullo stack)
Ad ogni variabile di tipo riferimento può essere assegnato il riferimento null (cioè puntatore a niente).

Gli oggetti sono sempre passati per riferimento, invece le variabili primitive per valore (copia)

Array

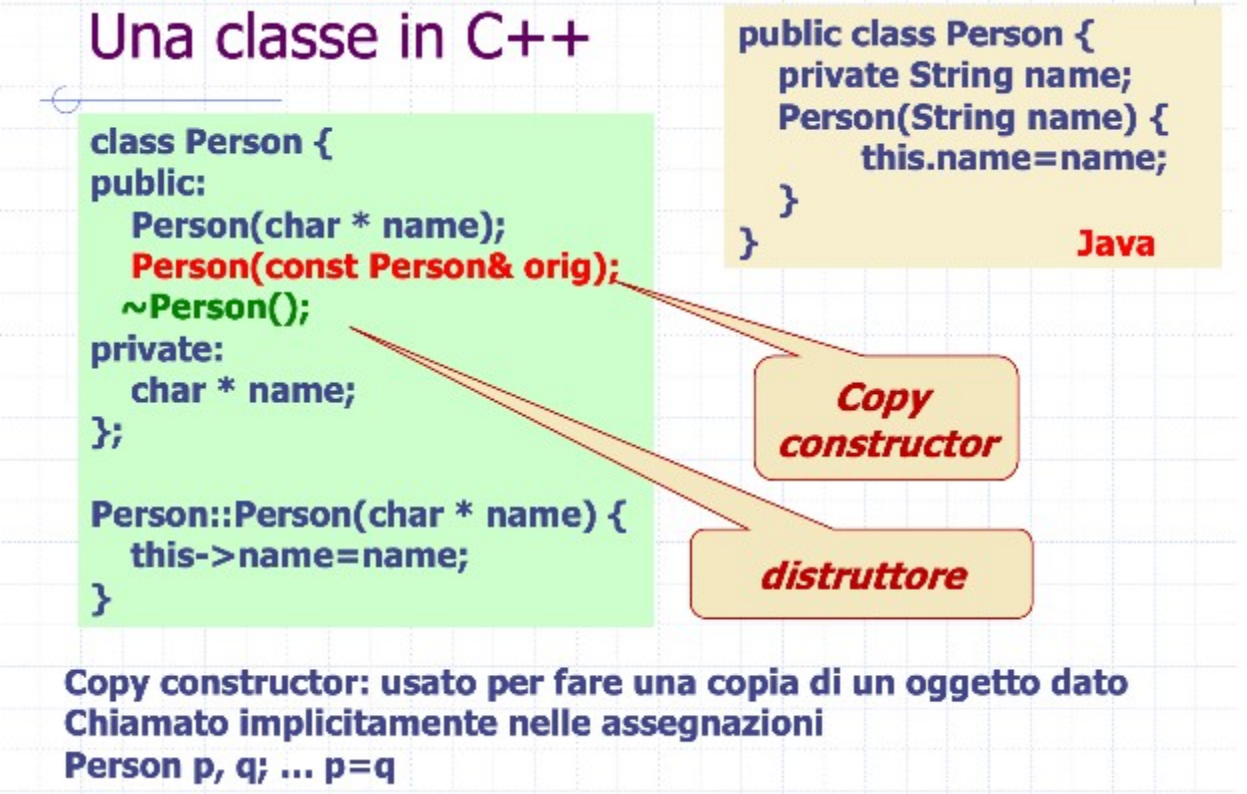
In Java anche gli array sono dei riferimenti, come gli oggetti.
In Java non esistono array statici, cioè non si possono allocare nello stack
Per coerenza anche in Java si usa la sintassi $oggetto.parametro$ perchè dichiariamo le variabili come se non fossero riferimenti (anche se in realtà lo sono).
La sintassi è $tipo[]\ \ nomeArray;$ oppure $\int[]\ \ nomeArray = \{1,2,3\};$
Se manca l'inizializzazione non viene allocato lo spazio, l'allocazione si realizza dinamicamente tramite l'operatore $new\ \ <tipo>\ \ [<dimensione>]$
Se gli elementi non sono di tipo primitivo l'operatore new alloca solo lo spazio per i riferimenti



Una funzione utile disponibile è la copia di array:

$$System.arrayCopy(Object\ \ src, \int\ src_position, Object\ \ dst, \int\ dst_position, \int\ length);$$

System serve da libreria globale ma non è un oggetto, System è una classe con dei metodi statici.



Ereditarietà e polimorfismo

L'ereditarietà si ha quando per definire un oggetto partiamo da un altro preesistente e ci aggiungiamo qualcosa.

Ad esempio pila e coda sono molto simili come struttura dati, ma l'unica cosa che cambia è il metodo in cui si estraggono i dati. Quindi la differenza c'è solo nel metodo $estrai()$.
Per evitare di riscrivere tutto il resto di codice, che è uguale per entrambe le operazioni, posso fare copia e incolla e cambiare solo il metodo $estrai()$

Questa però non è una "buona pratica", per esempio se trovo un errore in una altra funzione, devo andare a risolvere su entrambe le implementazioni, e questo non è efficiente

Quindi esiste l'ereditarietà:
$public\ \ class\ \ Cosa\ \ extends\ \ Pila$
Che si può leggere come "la coda è una pila però…"
In pratica viene recuperato tutto il codice di Pila, e posso aggiungere e sovrascrivere ogni metodo.
In questo modo ho un codice più robusto (ad esempio per quanto riguarda l'esempio dell'errore di prima)

Gerarchia di ereditarietà

Tutte le classi di un sistema OO sono legate da una gerarchia di ereditarietà, di default ogni classe estende $Object$, che è la radice della gerarchia di ereditarietà
Java supporta solo l'ereditarietà semplice (una classe non può ereditare da più di un'altra classe), l'ereditarietà multipla invece permette di ereditare da più classi
Ogni classe Java implicitamente estende $Object$:

- $public\ \ boolean\ \ equals(Object);$
- $protected\ \ void\ \ finalize();$
- $public\ \ String\ \ toString();$

Ogni volta che faccio $System.out.println(oggetto)$ viene implicitamente stampato $System.out.println(oggetto.toString())$ il quale è dichiarato nella classe $Object$ (il quale stampa un id e altre cose) ma può essere sovrascritto nelle altre classi

Java si dice che abbia una struttura di tipo albero, perché tutte le classi estendono $Object$
In C++ invece non ci sono ereditarietà implicite, e infatti si dice che abbia una struttura di tipo foresta. C++ ha una ereditarietà multipla.

Ovviamente le estensioni possono essere sia strutturali (aggiunta variabili di istanza) e comportamentali (aggiunta e/o modifica (**overriding**) di metodi)

L'**overriding** consiste nel ridefinire metodi che precedentemente sono stati definiti nella classe padre, sovrascrivendoli (solo per il figlio ovviamente).
Con la keyword $super$ posso richiamare un metodo della classe padre, anche se è stato ridichiarato nella classa figlia

```
class AutomobileElettrica extends Automobile {
  ...
  void accendi() {
    if(batterieCariche) super.accendi();
    else System.out.println("Batterie scariche");
  }
}
```

I costruttori **non** vengono ereditati (anche perchè essendo in classi diverse, con nomi diversi, avranno nomi diversi) però si può richiamare con $super$ posta come **prima** istruzione del nuovo costruttore
Se non viene chiamato esplicitamente il costruttore della superclasse, il compilatore inserisce automaticamente il codice che ne invoca il costruttore di default $super();$ (senza parametri)

Nelle assegnazioni bisogna avere sia a sinistra che a destra lo stesso tipo, però esistono delle eccezioni nei casi in cui non si perdono informazioni (ad esempio un int che diventa un float)
Per i tipi di dato non primitivo questa regola vale in base all'ereditarietà.
Ad esempio $classePadre\ \ x = new\ \ classeFiglio(\ldots);$ questo perchè la $classeFiglio$ ha al suo interno ogni metodo e variabile di $classePadre$ e quindi è tutto coerente, ovviamente non si

possono usare variabili e metodi specifici e solamente appartenente a $classeFiglio$, perchè a livello di compilazione x è un $classePadre$, e quindi darebbe errore in fase di compilazione, si può però usare un casting.

Overriding significa sovrascrivere una cosa che già esiste (ad esempio si fa l'overriding di un metodo della classe figlio rispetto alla classe padre)
Con $@Override$ scritto nella riga prima del metodo, esplicito il fatto che quel metodo esiste già in una superclasse e lo sto sovrascrivendo, questo aiuta il compilatore.
Per esempio se scrivo $@Override$ e scrivo $tostring()$ senza maiuscola, il compilatore si accorge che non esiste nella superclasse un metodo chiamato così e viene dato un errore

Esiste anche l'**overloading** che consiste nell'avere più metodi con lo stesso nome a patto che si distinguano per numero e/o tipo dei parametri (il tipo del valore di ritorno non basta per distinguere i due metodi, questo perché il metodo potrebbe non essere stato ancora dichiarato al momento della chiamata). L'oggetto che comprende il nome della funzione e il numero con il tipo dei parametri si chiama **firma** (ad esempio $nomeFunz(\int, String, char)$)

- Overloading
  Funzione con uguale nome e diversa firma che possono coesistere
- Overriding
  Ridefinizione di una funzione mantenendo immutata la firma e "cancellando" la definizione precedente

Costruttori

-Se per una classe non definisco nessun costruttore, il compilatore lo crea in automatico (senza parametri e inizializzando tutto)
-Se definisco **almeno** un costruttore non void (ovvero senza parametri), il sistema **non** crea in automatico il costruttore precedente.
-Se B è figlia di A, il costruttore di B come prima cosa invoca A(), a meno che la **prima** istruzione non sia $super(\ldots)$, cioè a meno che non ne chiamo io un altro esplicitamente
-Nel caso in cui io per una classe non definisco un costruttore void ma solo non void, e in una sottoclasse non metto come prima istruzione $super(parametri)$, viene dato errore perchè verrà chiamato $super()$, che non esiste
-Per chiamare un costruttore della stessa classe si usa $this(\ldots)$, e **deve** essere chiamato come prima istruzione del costruttore.

Binding

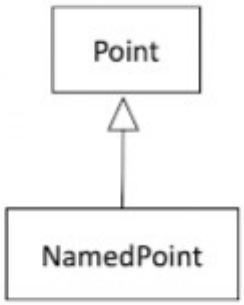
Polimorfismo: la capacità di assumere forme diverse, nella programmazione è
la capacità di un elemento di riferirsi a elementi di diverso tipo

Se S è un sottotipo di T, allora variabili di tipo T possono essere sostituite da variabili di tipo S senza alterare alcuna proprietà desiderabile del programma

```
Point p=new Point(1,2);
p.move(3,4);
```

Ovunque ci sia un Point posso mettere un NamedPoint

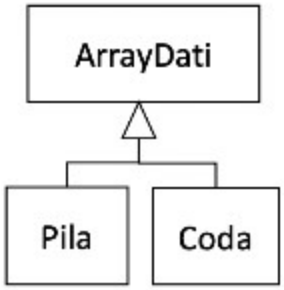
```
Point p=new NamedPoint(1,2,"A");
p.move(3,4);
```



Una variabile di tipo riferimento T può riferirsi ad un qualsiasi oggetto di tipo T **o un suo sottotipo**

Bisogna distinguere le assunzioni che si fanno a compile-time e quelle che si fanno a run-time.

```
public static void main(String
a[]){
    ArrayDati p;
    // leggi k
    if (k==1) p = new Pila();
    else p = new Coda();
    p.inserisci(1);
    p.inserisci(2);
    p.estrai();
}
```



A compile-time p sarà sempre di tipo $ArrayDati$, invece a run-time viene deciso durante l'esecuzione, quindi potrà essere una $Pila$ o una $Coda$.
Questa strategia viene chiamata **dynamic binding** (oppure late binding, o anche lazy evaluation)
In C++ invece di default c'è il static binding, però è possibile scegliere che un metodo si comporti come binding dinamico con la keyword *virtual*
In Java abbiamo un static binding a compile-time e abbiamo un dynamic binding a run-time (esistono casi extra come private,static e final)

Regole

1. Il compilatore determina la firma del metodo da eseguire basandosi sempre sul **tipo statico**
2. In caso di overriding (e solo in questo caso), la specifica implementazione del metodo la cui firma è stata decisa dal compilatore viene determinata a run-time sul **tipo dinamico**

Le firme vengono scelte a compile-time, e a run-time si scelgono con l'ereditarietà in base alle firme scelte a compile-time.

```
class A {
    public void g(A x) {System.out.println("called 0 on instance of A");}
}
class B extends A {
    public void g(A x) {System.out.println("called 1 on instance of B");}
    public void g(B x) {System.out.println("called 2 on instance of B");}
}
```

```
A a=new A();
A ab=new B();
B b=new B();
```

a.g(a); => A.g(A) => A.g(A)	called 0 on instance of A
a.g(ab); => A.g(A) => A.g(A)	called 0 on instance of A
a.g(b); => A.g(A) => A.g(A)	called 0 on instance of A
ab.g(a); => A.g(A) => B.g(A)	called 1 on instance of B
ab.g(ab); => A.g(A) => B.g(A)	called 1 on instance of B
ab.g(b); => A.g(A) => B.g(A)	called 1 on instance of B
b.g(a); => B.g(A) => B.g(A)	called 1 on instance of B
b.g(ab); => B.g(A) => B.g(A)	called 1 on instance of B
b.g(b); => B.g(B) => B.g(B)	called 2 on instance of B

Classi astratte

Un metodo astratto è un metodo per il quale non è specificata alcuna implementazione.
Quando una classe viene dichiarata astratta (con la keyword *abstract*) non si possono creare istanze "dirette" di quella classe, invece si possono creare istanze di sue sottoclassi (è fatta apposta).
Se una classe ha almeno un metodo astratto (o ne eredita uno senza implementarlo), allora la classe **deve** essere astratta
Per poter usare correttamente una sottoclasse di una classe abstract devo implementare e definire ogni metodo che è astratto nella superclasse.
Per il principio di ereditarietà è corretto scrivere $classePadreAstratta \;\; x = new \;\; classeFiglio(\ldots);$ purché nella classe figlio vengano sovrascritti i metodi astratti della classe padre

Information hiding

La visibilità degli attributi e metodi di una classe possono essere (in ordine dal più visibile al meno visibile):

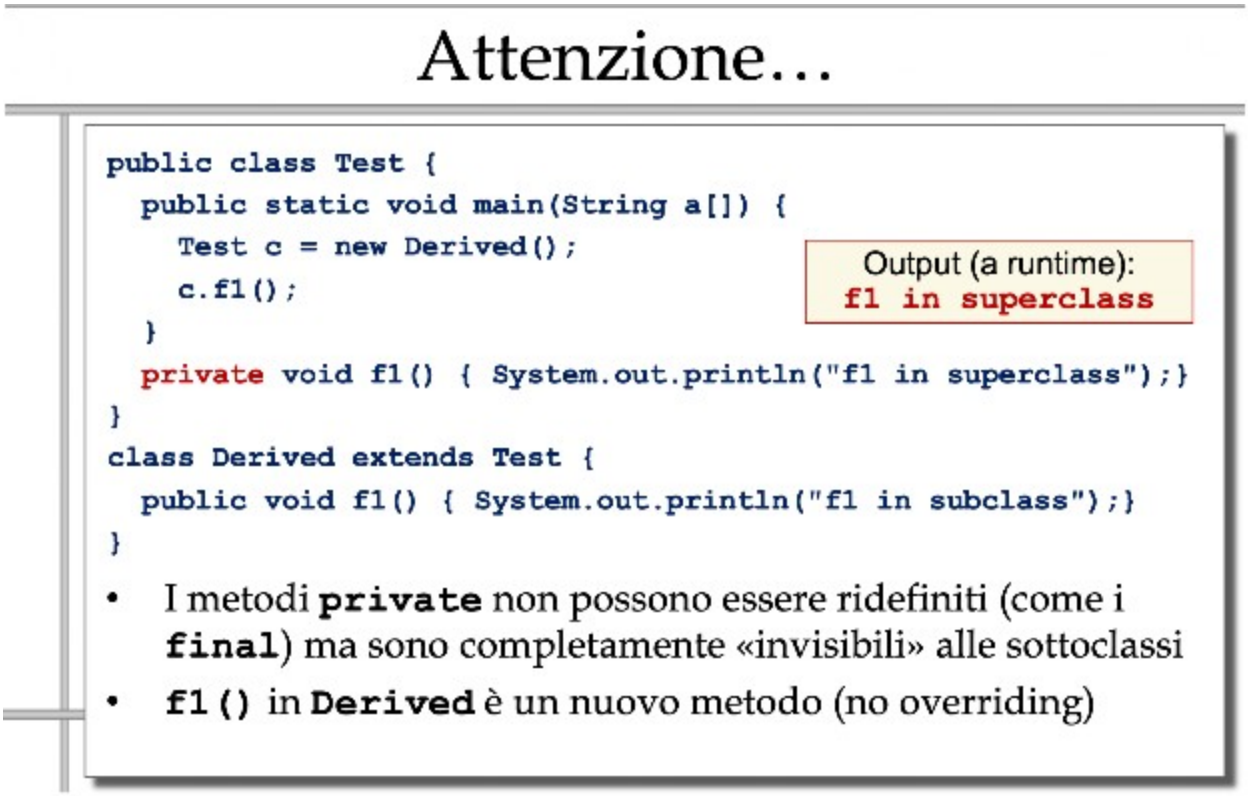
- public
  visibili a tutti, vengono ereditati
- protected
  come package ma sono visibili anche alle sottoclassi in package diversi
  vengono ereditati
- package (nessun modificatore)
  visibili solo al package
- private
  visibili solo all'interno della classe
  non visibili nelle sottoclassi

	Class	Package	Subclass (same pkg)	Subclass (diff pkg)	World
`public`	+	+	+	+	+
`protected`	+	+	+	+	
no modifier	+	+	+		
`private`	+				

Per impedire la creazione di sottoclassi di una certa classe, dobbiamo dichiarare questa classe definendola *final*.
Questa logica si può usare anche per quanto riguarda i metodi, per impedire di eseguire l'overriding, e questo si fa sempre definendolo *final*.
Quindi per ora abbiamo **due usi** di *final*, per le costanti e per cose su cui non vogliamo si possa fare override o estensione.
Ad esempio la classe *string* è dichiarata *final*, per una questione di binding, perché così facendo si ottimizza evitando il dynamic binding (non cerca appena a run-time metodi di sottoclassi) perché viene fatto il static binding.
In Java viene eseguito sempre il dynamic binding, tranne quando si può decidere automaticamente a compile-time (nei casi di metodi private, static e final e nel caso dei costruttori)

Se io dichiaro privato un metodo nella superclasse e lo ridichiaro public nella sottoclasse, viene fatto nuovamente binding statico, chiamando quello della superclasse, questo perchè non è un overriding essendo che il metodo final della superclasse è invisibile alla sottoclasse.
Questo succede perché *private* ha lo stesso effetto di *final*, ovvero abilita lo static binding, solo che in questo caso la sottoclasse non vede che esiste questo metodo, quindi non c'è nessun

errore. Infatti se l'oggetto fosse stato anche staticamente *Derived*, avrebbe chiamato il metodo della sottoclasse.



Un uso di *final* potrebbe essere ad esempio quando dichiaro *ArrayDati* per poi fare *Pila extends ArrayDati*, mettendo final tutti i metodi tranne quello astratto *estrai*, cosi facendo impedisco di modificare in *Pila* gli altri metodi di *ArrayDati* che voglio restino uguali.

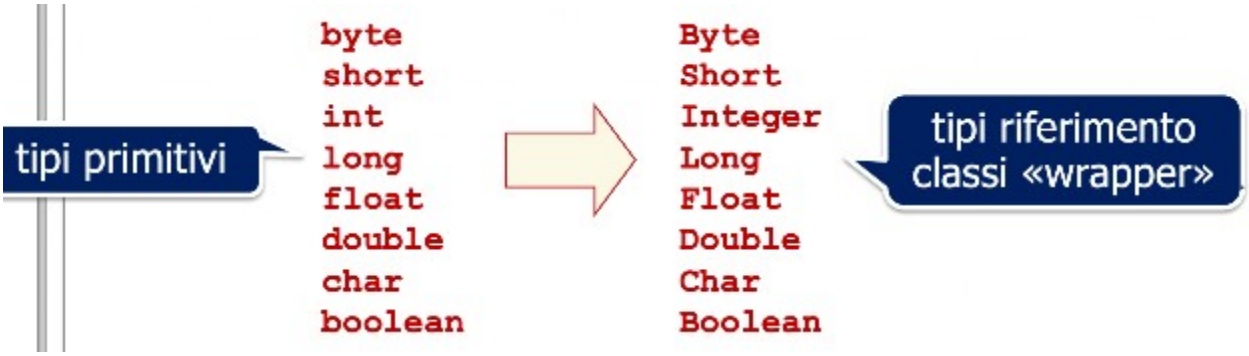
Per tutti i tipi primitivi esistono delle classi "wrapper" che rendono anche i tipi di dati primitivo sottoclassi di *Object*, questo aiuta nel caso di dati non definiti (per cui si usa appunto *Object* per comodità).
Per inserire un intero all'interno di una collection devo usare la sua classe wrapper *Integer*.

Una Pila polimorfa …

```
public static void main(String args[]) {
  int dim = 10;
  Pila s = new Pila();
  for(int k=0; k<dim; k++){
    Integer o = new Integer(k);
    s.inserisci(o);
  }
  for (int k=0; k<3*dim; k++) {
    Integer i = (Integer) s.estrai();
    int w = i.intValue();
    System.out.println(w);
  }
}
```

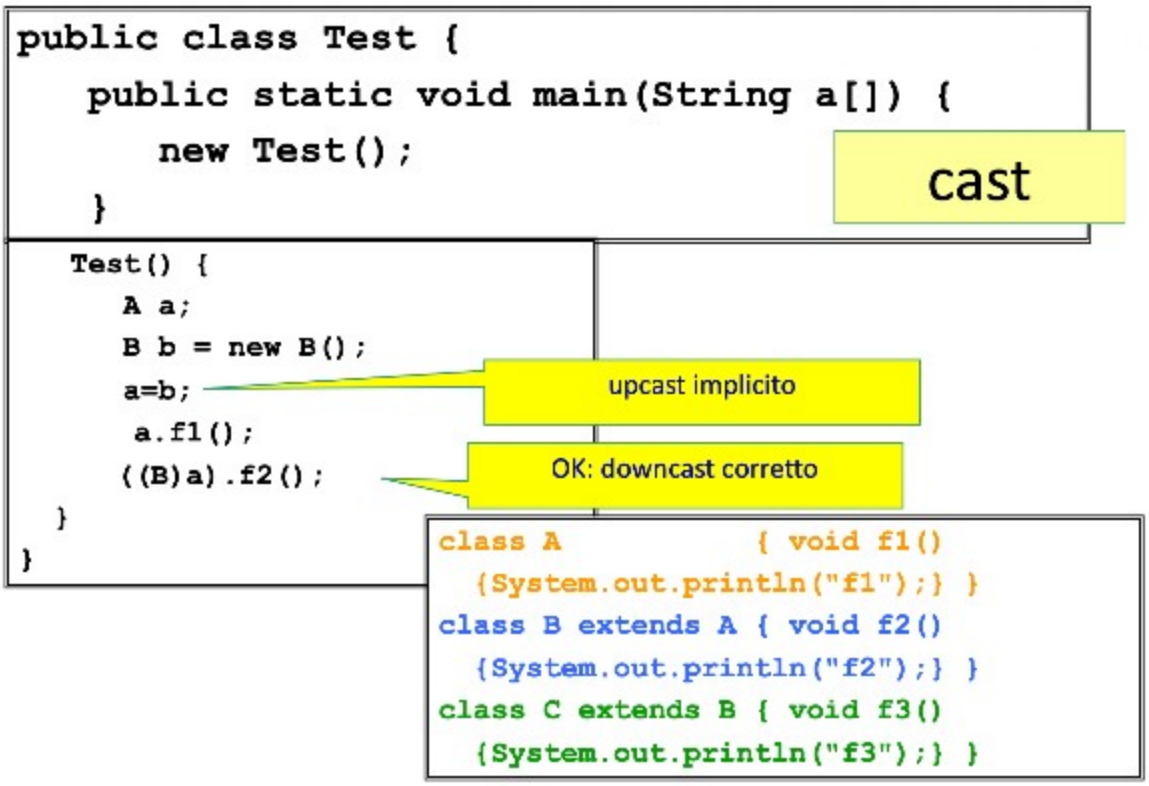
Essendo che la pila è di *Object*, possiamo metterci dentro qualsiasi oggetto e non solo *Integer*, quindi sarebbe più corretto assicurarsi che quello che stiamo leggendo sia un *Integer*

Java questa cosa la fa in automatico, quindi quando si aspetta un *Object* e vede una variabile di tipo primitivo esegue il cosiddetto **autoboxing**, e casta in automatico l'intero in *Integer*.
Quindi ad esempio possiamo scrivere *Character ch='a'*
Questa cosa funzione anche per il contrario, quindi per tornare da *Integer* a intero, e viene detto **unboxing**.



Esiste anche il casting tra classi, che a differenza di casting tra tipi primitivo (che converte la variabile) il casting tra oggetti serve per "comunicare" al compilatore se un dato a runtime sarà *Integer* piuttosto che *Object* ad esempio.
Nel caso in cui a runtime non venga "rispettato" il casting viene dato un errore
class cast Exception

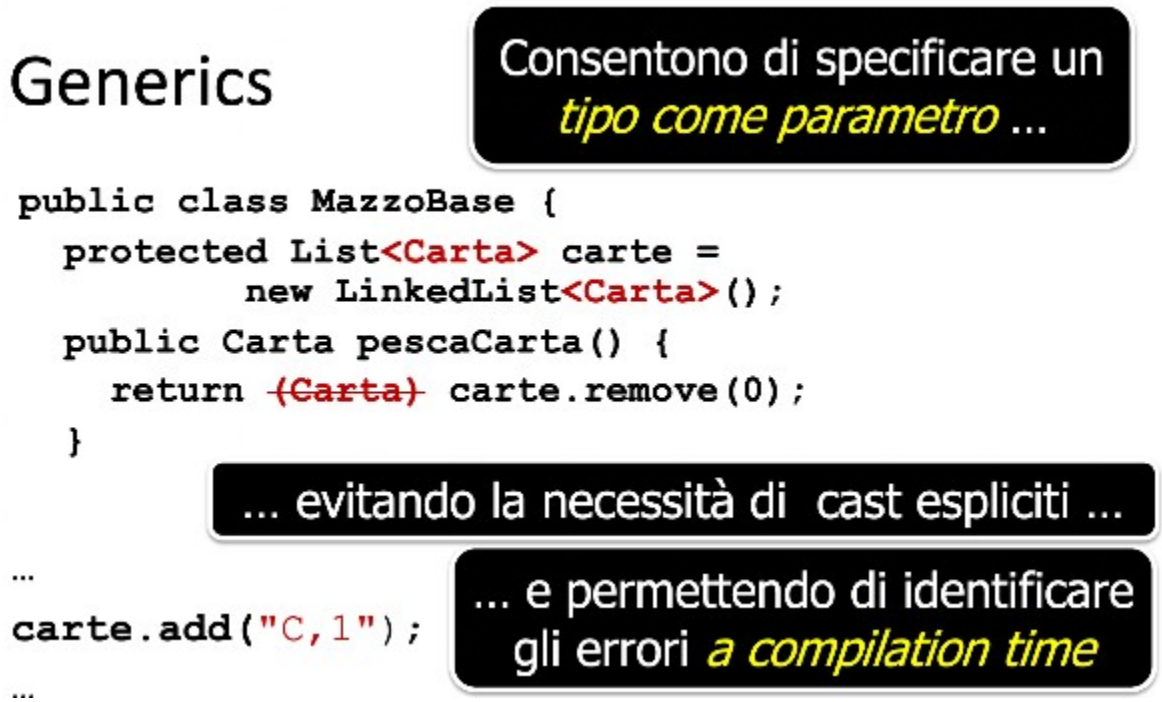
Questo casting si svolge solo al contrario della idea *is-a*
Queste due operazioni si chiamano upcast (is-a) e downcast (casting esplicito)
Il downcast è possibile solo se il tipo dinamico dell'oggetto convertito è un sottotipo dell'oggetto a sinistra dell'uguale



Esistono però dei casi in cui io non so che tipo runtime ci sia dentro ad un *Object*, in questo caso per fare il cast bisogna prima controllare con un if che cosa c'è dentro a quell'oggetto, questo si fa usando l'operatore *instanceof* (ad esempio *o instanceof Integer*)

Generics

Quando ho una classe di tipo List tipo LinkedList, essendo che comprendono object, possiamo definire un sottoinsieme sul quale opera, usando *new LinkedList<Carta>*, così non serve fare il cast durante l'estrazione e ci saranno i controlli che non vengano inseriti oggetti che non siano di tipo (o sottotipo) Carta

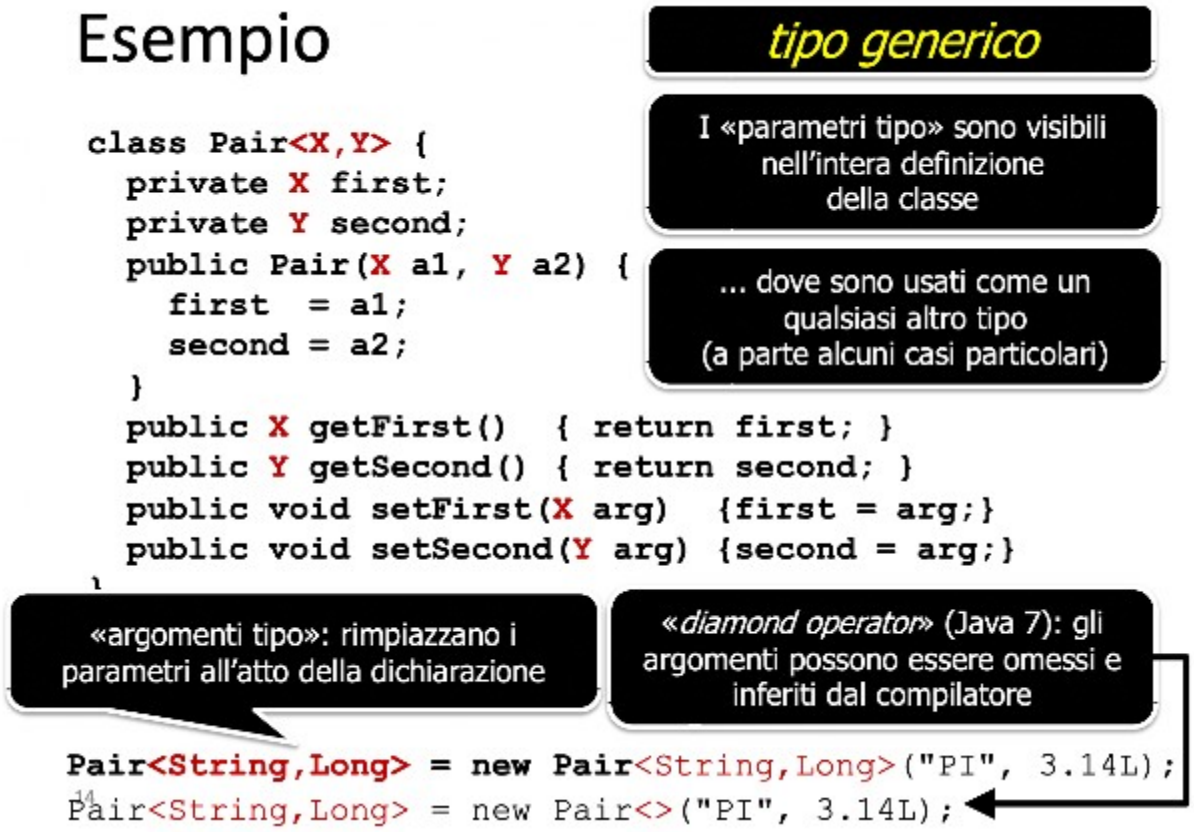


Per definire una classe (o una interfaccia) che accetta tipi generici dobbiamo definirlo similmente all'uso, quindi:

$new\ \ Name\ \ <T1,T2,\ldots,Tn>\{\};$

Il tipo attuale deve essere specificato nella dichiarazione:

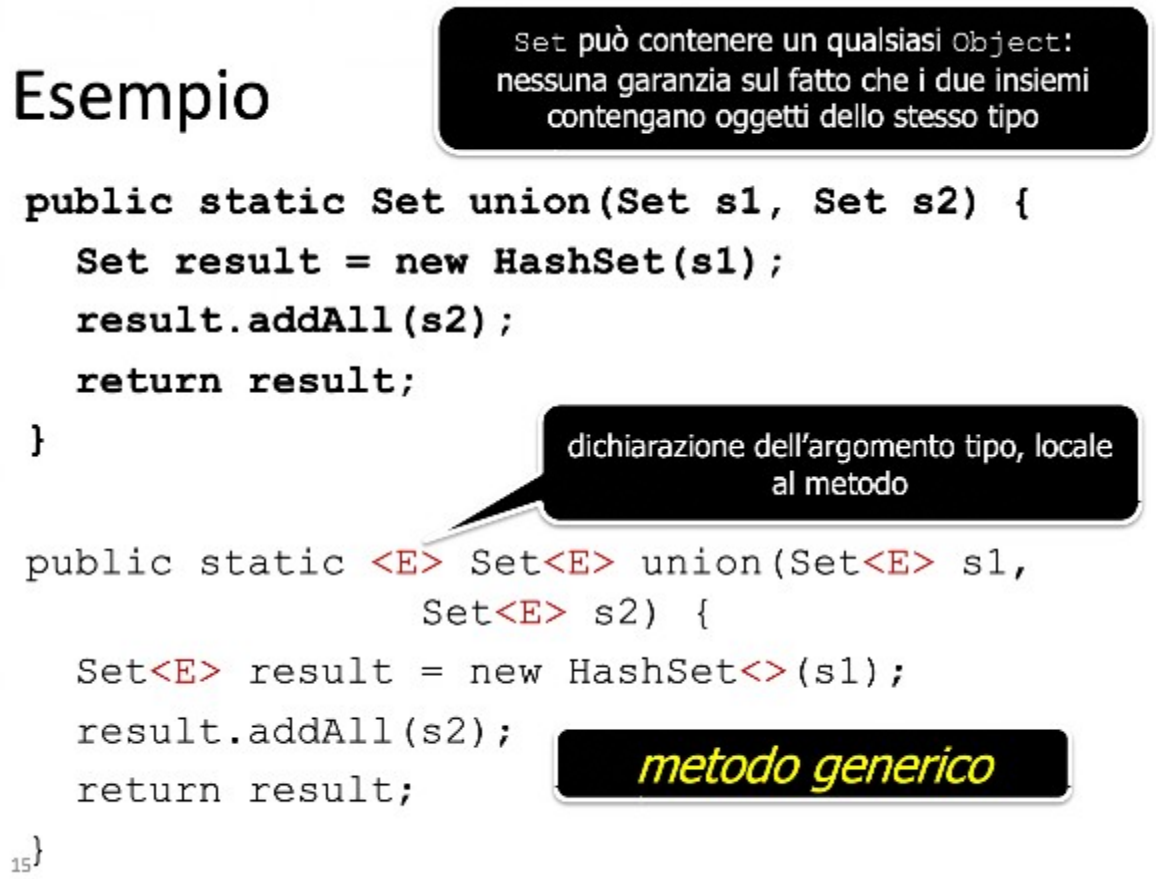
$Name<Persona>\ \ nome\ \ =\ \ new\ \ \ Name<Persona>;$



Oppure sfruttando la **diamond operator** (gli argomenti possono essere omessi e inferiti dal compilatore):

$Name<Persona>\ nome\ =\ new\ \ Name<>;$

Si possono anche definire dei metodi generici, specificando prima di tutto che abbiamo un parametro generico:



Il Java Collection framework fa sempre uso dei tipi generici

Dati due tipi generici $G<A>$ e $G<B>$ dove B è una sottoclasse di A, **non è vero** che $G<B>$ è una sottoclasse di $G<A>$

Questo perché i generics in Java sono implementati in maniera che l'informazione sui parametri viene eliminata dopo i controlli statici e vengono sostituiti con degli opportuni cast.

Esempio

```
class Pair<X,Y> {
  private X first;
  private Y second;
  public Pair(X a1, Y a2) {
    first  = a1;
    second = a2;
  }
  public X getFirst()  { return first; }
  public Y getSecond() { return second; }
  public void setFirst(X arg)  {first = arg;}
  public void setSecond(Y arg) {second = arg;}
}
class Test {
  public static void main(String[] args) {
    Pair<String,Long> pair =
      new Pair<>("PI", 3.14L);
    String s = pair.getFirst();
    Long   l = pair.getSecond();
    Object o = pair.getSecond();
  }
}
```

```
public class Pair {
  private Object first;
  private Object second;
  public Pair(Object a1,
              Object a2) {
    first = a1;
    second = a2;
  }
  public Object getFirst() { return first; }
  public Object getSecond() { return second; }
  public void setFirst(Object arg) { first = arg; }
  public void setSecond(Object arg) { second = arg; }
}
final class Test {
  public static void main(String[] args) {
    Pair pair = new Pair("PI", 3.14L);
    String s = (String) pair.getFirst();
    Long   l = (Long)   pair.getSecond();
    Object o =          pair.getSecond();
  }
}
```

Come vediamo, in realtà viene tutto convertito in quello che avremmo fatto senza tipi generici, ma ricordiamo che i tipi generici ci aiutano per quanto riguarda errori e compilatore

Se usiamo in una funzione una lista generics, non sappiamo che tipo generico assumerà la lista durante la compilazione, e non essendoci l'is-a tra Collection di tipi non posso avere come parametro della funzione Object.
Quindi posso nei parametri del metodo inserire una cosiddetta **wildcard** così facendo:
$static \ \ void \ \ printList(List<?> \ \ list)\{\};$

Per poter restringere gli oggetti definiti da una wildcard, posso usare $<? \ \ extends \ \ Persona>$ per dire che $?$ potrà essere una sottoclasse di Persona, questa cosa può essere usata anche con le interfacce (sempre con extends), oppure combinando una classe con più interfacce (non più classi perché non c'è ereditarietà multipla)

Generics e sottotipi: *bounded wildcards*

- È possibile «limitare» le wildcard specificando che sono accettati solo sottotipi di un tipo dato

```
Group<? extends Persona> g
            = new Group<Student>();
```

- Si usa **extends** anche con le interfacce

```
Group<? extends Comparable> g
            = new Group<Student>();
```

- È possibile specificare più di un tipo: se c'è una classe deve essere la prima

```
Group<? extends Persona & Comparable> g
            = new Group<Student>();
```

22

Si possono fare queste limitazioni anche al rovescio, ovvero riferendosi alle superclassi, usando *<? super Persona>*

In generale non è possibile costruire degli array di generics.

I generics non valgono con l' instanceof, questo perchè a runtime la lista non ha più l'informazione di che tipo è, proprio per la sostituzione di codice di cui abbiamo parlato prima. Quindi non possiamo fare confronti su liste tipizzate.

- Generics non si possono usare con **instanceof**

```
Object obj = new LinkedList<Long>();
obj instanceof List                    //true
obj instanceof List<?>                 //true
obj instanceof List<Long>              //error
obj instanceof List<? extends Number>  //error
obj instanceof List<? super Number>    //error
```

Interfacce

Un'interfaccia è una classe completamente astratta, senza attributi, quindi è formata solo da una collezione di firme di metodi **pubblici** e **astratti**
Le interfacce non hanno un costruttore e infatti non possono essere istanziate (usate per creare un oggetto). Si possono però avere delle variabili di un'interfaccia le quali vengono istanziate con classi che implementano l'interfaccia

In certi casi si usano interfacce completamente vuote, usate solo per **etichettare** classi con speciali proprietà

Sintassi: $interface \ < nomeclasse > \{\}$

Un'interfaccia può ereditare da **una o più** interfacce, ma mai da classi
Il problema che si ha con le classi quando si utilizza ereditarietà multipla sono quali variabili d'istanza far valere e che metodo con la stessa firma implementare.
Per quanto riguarda le interfacce non ci sono questi problemi, perchè non ho variabili di istanza e i metodi non hanno implementazione, quindi prendendo solo la firma, non conta la sovrapposizione di metodi

Una classe può implementare **una o più** interfacce, e **deve** implementarne tutti i metodi (a meno che non sia una classe astratta)
I metodi di un'interfaccia sono di default astratti.

Si può vederla così: una classe definisce che un oggetto **è** qualcosa; un interfaccia invece definisce che un oggetto **si comporta** in una certa maniera.

Si possono usare variabili di tipo interfaccia a sinistra di un'uguaglianza, ma non si possono usare a destra (perchè non si possono istanziare).

Conversioni

Da stringhe a numeri

Per conversioni "lecite" si usa $var \ \ i \ = \ Var \, . \, parseVar (string) ;$

Errori

Bisogna inserire il codice "pericoloso" (cioè che potrebbe generare un errore) all'interno di un blocco protetto, con la sintassi $try \{ \ldots \} catch (tipoErrore \ \ ex) \{ \ldots \} ;$
Si può aggiungere anche $finally$ dopo $catch$ per inserire del codice da fare a prescindere (sia in caso di errore sia no)

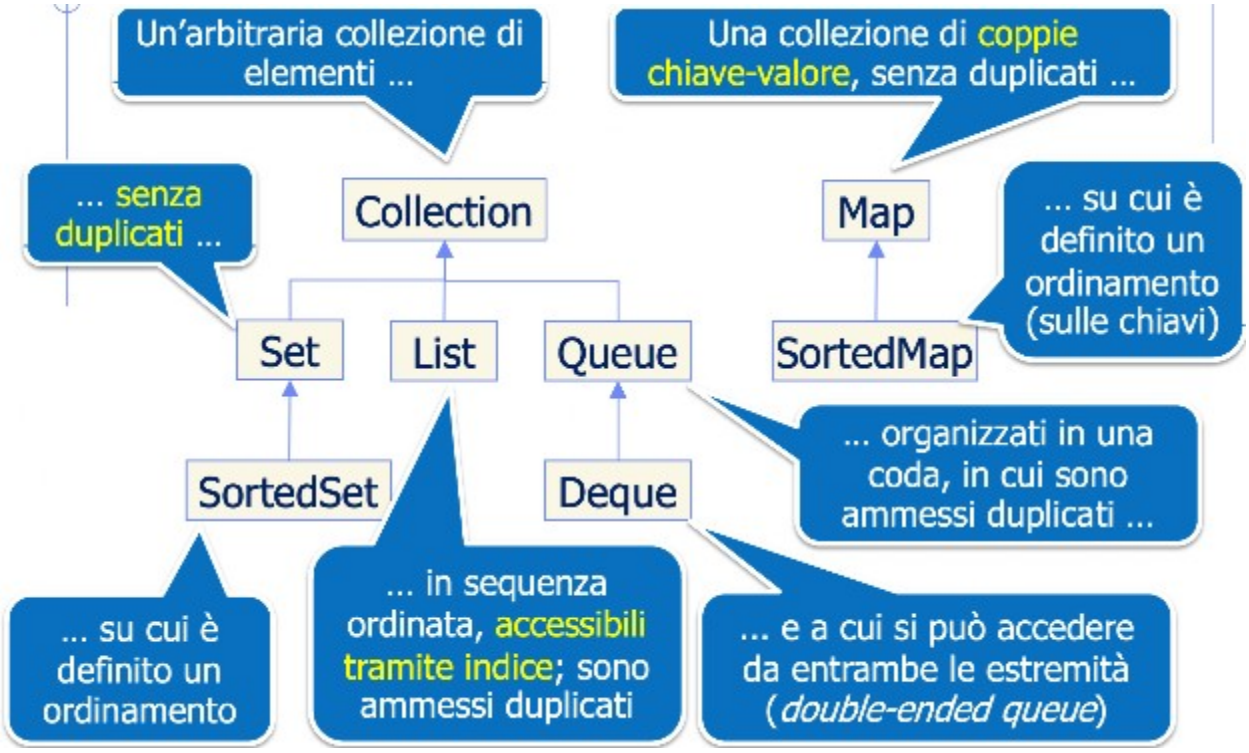
Collections

Una **collection** è un oggetto che raggruppa elementi multipli in un'unica entità
Le collection possono contenere oggetti di tipo diverso, a differenza degli array che devono essere omogenei.
Vengono usate per immagazzinare, recuperare e trattare dati, e soprattutto per trasferire gruppi di dati da un metodo all'altro

Il **Collections Framework** contiene tre tipi di elementi: interfacce, implementazioni(classi) e algoritmi.
Le collection sono utili per ridurre il lavoro del programmatore che si può concentrare sullo sviluppo ottimizzato dell'applicazione invece di "perdere" tempo sullo sviluppo di strutture dati, inoltre le collection facilitano il riutilizzo del software

Per usare il Java Collections Framework bisogna implementare $java.util.*$

Le interfacce sono le seguenti:



È buona pratica implementare solo le librerie effettivamente utilizzate
Esistono delle operazioni di base comuni per tutte le Collection:

- int size();
- boolean isEmpty();
- boolean contains(Object e);
- boolean add(Object e);
- boolean remove(Object e);

- Iterator iterator();

La maggior parte di queste funzioni possono essere usate con *All* alla fine per passare come parametro un'altra collection invece di un singolo elemento.

Esiste il metodo *toArray*( ); che permette di ottenere il contenuto di una Collection sotto forma di array (utile nel caso in cui dobbiamo utilizzare delle API basate su array invece che Collections)

List: operazioni base (addizionali)

```
Object get(int index);
Object set(int index, Object element);
void add(int index, Object element);
Object remove(int index);
int indexOf(Object o);
int lastIndexOf(Object o);
```

Map: associazione chiave-valore

La chiave deve essere univoca!

Operazioni base:

```
Object put(Object key, Object value);
Object get(Object key);
Object remove(Object key);
boolean containsKey(Object key);
boolean containsValue(Object value);
int size();
boolean isEmpty();
Set keySet();
Collection values();
```

Queste sono alcune implementazioni delle Collections

		classi (implementazioni)			
		hash table	resizable array	balanced tree	linked list
interfacce	Set	HashSet		TreeSet	
	List		ArrayList		LinkedList
	Map	HashMap		TreeMap	

La differenza tra ArrayList e LinkedList è che la prima si preferisce quando la struttura è relativamente stabile (quando gli elementi contenuti sono fissi e cambiano poco), invece la seconda quando si aggiungono e cancellano spesso elementi. È sempre consigliato usare interfacce

Nel HashSet gli elementi non sono ordinati, mentre in TreeSet si

Posso dichiarare cosa dovrà contenere una collection usando $<type>$, altrimenti essendo qualsiasi oggetto sottoclasse di Object, potrei mettere in una collection qualsiasi tipo di dato, riscontrando problemi nel momento in cui li estraggo.
Così facendo invece il compilatore mi da errore quando inserisco nella collection un tipo incompatibile, e durante l'estrazione dalla collection non devo fare il cast perchè sarà eseguito in automatico

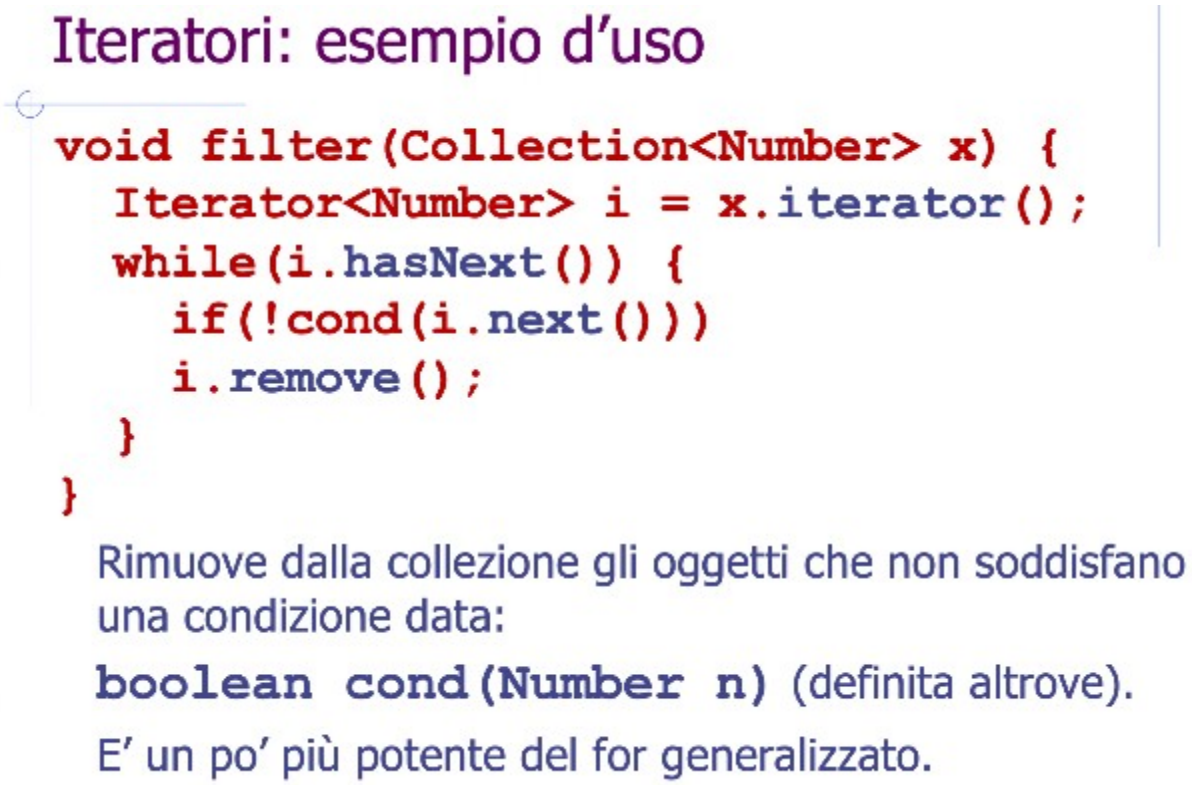
Posso creare un ciclo for generalizzato, cioè senza un indice ma direttamente scorrendo tutta la Collection dentro una variabile:

```
List<Number> listaDiNumeri=new LinkedList();
…
for (Number n : listaDiNumeri){
     System.out.println(n);
   }
```

Attenzione: non posso modificare
La struttura mentre la visito:
no remove()

Per poter rimuovere gli elementi non si può usare questo costrutto, ma posso usare un *Iterator*, a cui viene associata ogni Collection.
L'iterator permette di scandire gli elementi di una collection uno a uno





Se usiamo un set e ci aggiungiamo oggetti "uguali", nel senso con gli stessi attributi, notiamo che gli oggetti vengono ripetuti pur essendo un Set (il quale non ammette ripetizioni), questo perchè bisogna ridefinire e scrivere i metodi $equals()$ e $hashCode()$. Altrimenti il confronto viene fatto tra i riferimenti, che sono sempre diversi a meno che non si riferiscono allo stesso oggetto appunto.

Classi interne

Si possono creare classi interne ad altre classi e sono visibili solo nelle funzioni nelle quali vengono dichiarate, quindi lo scope è quello della funzione nella quale è stata dichiarata

Uguaglianza e identità

Se istanzio due oggetti dello stesso tipo con gli stessi identici valori per gli attributi, essi non saranno uguali con il solito operatore $==$.

Invece se faccio la stessa cosa con due variabili di tipo primitivo funzionerà correttamente.

Invece se istanzio un oggetto e lo inizializzo copiando da un altro oggetto $p2 = p1$. In questo caso l'uguaglianza darà risposta *true*. Questo succede perché i nomi degli oggetti sono dei riferimenti agli oggetti, quindi eguagliando con l'operatore $==$ due riferimenti sono sempre diversi a meno che non si riferiscano allo stesso oggetto.

Esiste però un metodo di *Object* chiamato $.equals()$, però anche questo darebbe *false* nel caso in cui due oggetti puntino a due oggetti diversi, perché al suo interno viene fatto $p1 == p2$
Il metodo giusto per eseguire l'uguaglianza è eseguire l'override di *equals* decidendo noi i criteri per il confronto.
Bisogna seguire certe regole per evitare errori:

1. Controllare che l'oggetto passato come parametro per il confronto non sia *null* (invece per l'oggetto su cui chiamo equals non devo fare controlli perchè non si possono chiamare metodi su oggetti *null*)
2. Bisogna mettere il tipo dell'oggetto del parametro come *Object*, altrimenti sto eseguendo overloading e non overriding
3. Controllare che l'oggetto che arriva come parametro sia un'istanza dell'oggetto corretto (l'operatore *instanceof* verifica se vale la *is-a* e non è detto che l'oggetto sia propriamente di quella classe) e devo decidere io se far valere la equals anche con sottoclassi, in caso voglio che non valga per le sottoclassi devo usare $o.getClass()$

Per poter usare la $.equals()$ solo per oggetti della stessa classe del chiamante bisogna usare la $getClass()$ così da essere sicuri che siano la stessa classe.

Quale soluzione scegliere?

```
if (o.getClass() != this.getClass())
   return false;

oppure

if (!(var instanceof P)) return false;

?

Dipende...
```

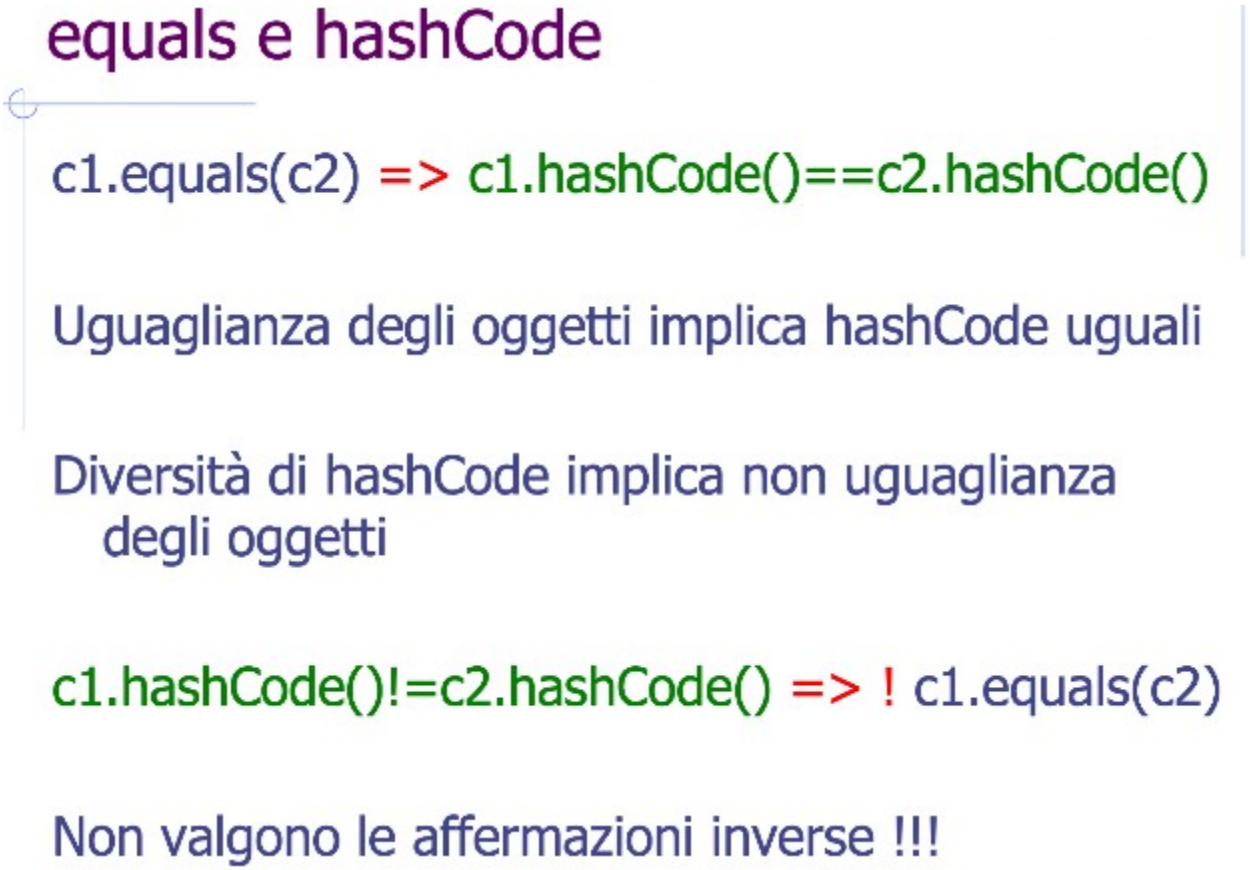
Ogni volta che si sovrascrive il metodo $.equals()$ bisogna essere **certi** di aver implementato il metodo rispettando le seguenti proprietà:

Proprietà di `equals()`

- Il metodo `equals()` implementa una relazione di equivalenza fra elementi non nulli e deve soddisfare le seguenti proprietà:
  1. riflessiva: per ogni riferimento non nullo `x`, `x.equals(x)` ritorna `true`
  2. simmetrica: per ogni riferimento non nullo `x` e `y`, `x.equals(y)` ritorna `true` se e solo se `y.equals(x)` ritorna `true`
  3. transitiva: per ogni riferimento non nullo `x`, `y`, `z`, se `x.equals(y)` e `y.equals(z)` ritornano `true` allora `x.equals(z)` ritorna `true`
  4. consistente: per ogni riferimento non nullo `x` e `y`, invocazioni diverse di `x.equals(y)` ritornano lo stesso valore, se nessuna delle informazioni usate da equals() sono state modificate
  5. per ogni riferimento non nullo `x`, `x.equals(null)` ritorna `false`

L'IDE può generare in automatico il metodo equals corretto, visto che è sempre uguale, indicando quali attributi utilizzare per il confronto.
Ogni volta che si scrive la $.equals()$ bisogna (**obbligatorio**) anche sovrascrivere il metodo $\int hashcode()$.
Il metodo $hashcode()$ mappa un oggetto su un intero, ovvero dato in input un oggetto, viene dato in output un intero, è una funzione non iniettiva (non invertibile) e sono possibili collisioni, cioè più oggetti (diversi) possono dare lo stesso hash.

Abbiamo dunque che se due oggetti sono uguali (tramite $equals()$), allora saranno uguali anche gli hash, il viceversa invece non è detto sia vero.



Quindi lo hashcode può essere usato come meccanismo di “fail quick”, quindi prima controllo se gli hashcode sono diversi, e in quel caso so per certo che i due oggetti sono diversi, invece se gli hashcode sono uguali, devo fare il controllo con l’equals perchè potrebbero essere uguali o diversi

hash significa tritato (to hash=tritare)
L’hash può servire ad esempio per implementare un dizionario, creando un array in cui in ogni posizione associa tutti gli oggetti con hashcode equivalente all’indice di quella posizione, e così facendo velocizzo notevolmente la ricerca, vedendo subito se un elemento non appartiene al dizionario e potendo cercare in un insieme con meno elementi l’elemento.

Se non so come scrivere la hashcode, si può definirne una che torni sempre il valore 0 (anche se super inefficiente, basta definirla)

Proprietà di `hashCode()`

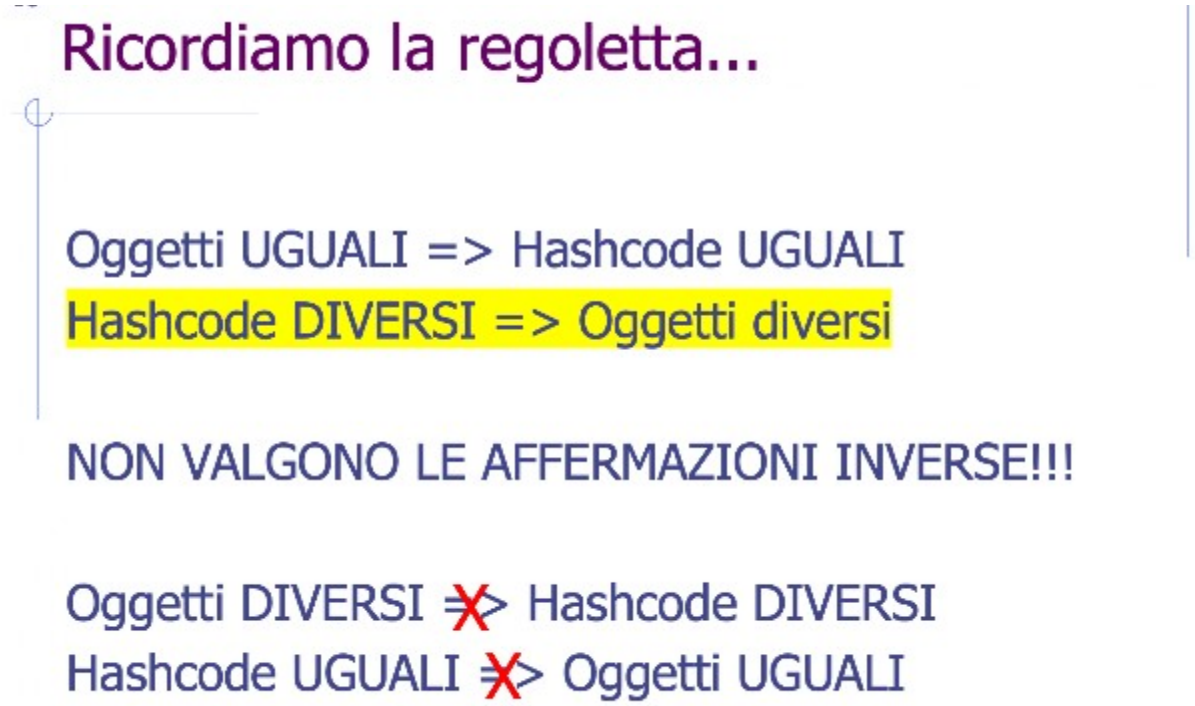
1. Se invocato più di una volta sullo stesso oggetto deve ritornare lo stesso intero
   - questo può essere diverso in esecuzioni diverse dell'applicazione; l'importante è che rimanga identico all'interno di una singola esecuzione
2. se due oggetti sono uguali secondo il metodo `equals()` allora `hashCode()` deve ritornare lo stesso intero
3. non è richiesto che a due oggetti diversi (secondo `equals()`) siano associati due `hashCode()` diversi
   - Tuttavia, questo in generale migliora le performance in alcune strutture dati («hash-based»)

Se il metodo viene ridefinito, garantire queste proprietà è compito del programmatore

Java fornisce un metodo per poter calcolare un hash in maniera predefinita, e devo passargli i parametri dell'oggetto che andranno a distinguere gli elementi nella equals:

$$return\ \ Objects.hash(var1, var2, \ldots)$$

Se non si implementa la hashcode, tutti gli oggetti saranno diversi, essendo che la hashcode verrà fatta sull'indirizzo degli oggetti e quindi non verrà controllata la equals implementata da noi. Quindi bisogna definire **sempre** la equals e la hashcode



Confronto

L'interfaccia *Comparable*, che bisogna implementare nella classe in cui c'è bisogno di eseguire il confronto, consente di definire un **ordinamento totale** fra gli oggetti che la implementano

Il metodo usato per il confronto è $\int \ compareTo(Object \ \ o)$ e restituisce:

- un intero negativo se *this* è minore di *o*
- un intero positivo se *this* è maggiore di *o*
- 0 se *this* è uguale a *o*



Solo gli oggetti che implementano *Comparable* possono far parte di un *SortedSet* o altre collection che fanno uso di ordinamento, questo perchè per oggetti generici non si ha un criterio di ordinamento, invece per classi che implementano *Comparable* esiste sicuramente l'implementazione del metodo *compareTo* (che è astratto e deve essere implementato).
Una collection di tipo $List$ o $array$, che contiene elementi comparabili, può essere ordinata facilmente usando $Collections.sort(o)$ e $Arrays.sort(o)$

Esiste poi l'interfaccia *Comparator* che permette di definire più criteri diversi e separati per il confronto, per far ciò permette di delegare il confronto a una classe separata e fornisce il metodo $\int compare(Object\ o1, Object\ 02)$ e quindi devo definire una nuova classe per ogni nuovo criterio, ognuna delle quali implementa *Comparator* e sovrascrive il metodo $compare()$.
Usare una classe *Comparator* è obbligatorio quando il criterio che si vuole usare è diverso da quello "naturale" implementato con *Comparable*.

In questa situazione quando vogliamo eseguire l'ordinamento, dobbiamo passare anche un'istanza del *Comparator* che vogliamo usare.

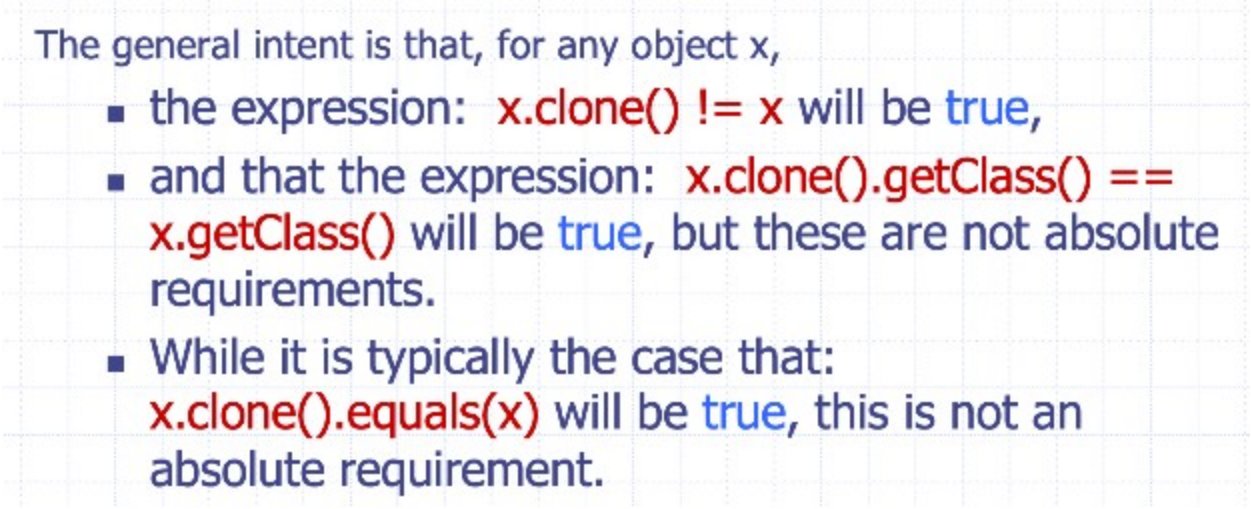
Clonazione

Se facciamo una copia ci aspettiamo che l'uguaglianza $x==clone$ dia false e invece $x.equals(clone)$ dia true.

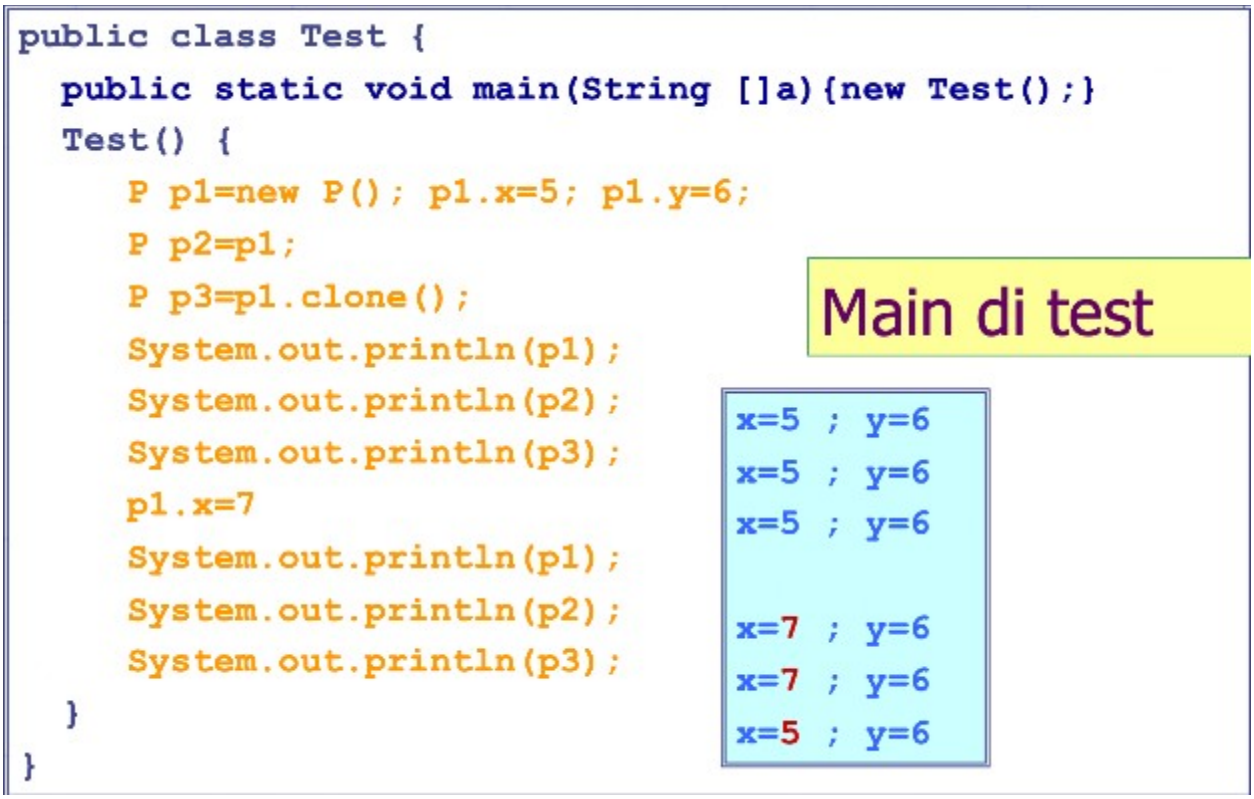
Quando faccio una copia con l'uguale di due oggetti, semplicemente copio l'indirizzo, così facendo qualsiasi modifica applicata ad uno dei due "nomi" verrà effettuata anche nell'altro (essendo lo stesso oggetto).
Ad esempio se voglio creare una copia di pila, devo scorrermi tutta la pila e inserire elemento per elemento nella nuova pila, così che siano uguali a livello di contenuto ma diverse in memoria.
La copia può essere semplicemente del **riferimento**. La copia può essere **superficiale** se effettivamente mi serve che un elemento sia "condiviso" tra altri (copia del riferimento degli oggetti interni all'oggetto principale). La copia invece può essere **profonda** se faccio una copia definita come prima (clonazione).
Esiste un metodo di Object che esegue la copia dell'oggetto:

$$protected \;\; Object \;\; clone()$$

Anche se la "copia" dell'oggetto può avere diversi significati, dipendenti dalla classe dell'oggetto



Il metodo $clone()$ però è protetto, quindi non posso chiamarlo fuori dalla sua classe.
Per poter usare al meglio questa funzione devo implementare la classe dell'oggetto con $Cloneable$. Infatti questa classe implementa un metodo $public \;\; Object \;\; clone()$ che al suo interno esegue $super.clone()$, il quale può essere chiamato essendo all'interno di una sottoclasse di Object.
Il metodo $clone()$ controlla che l'oggetto su cui è stato chiamato sia sottoclasse di $Cloneable$

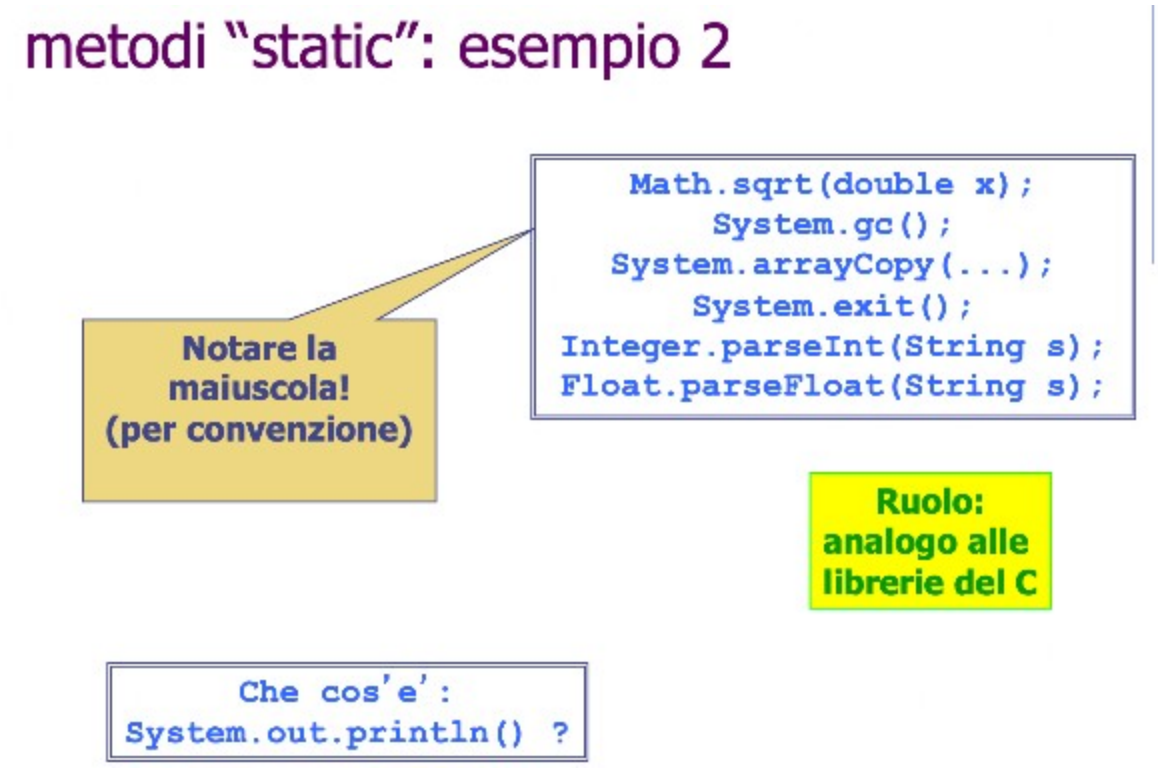


Nel caso in cui l'oggetto su cui facciamo clone abbia altri oggetti al suo interno, ovvero delle referenze ad oggetti (oggetti, array etc), la copia sarà superficiale, perchè vengono copiati i riferimenti degli oggetti interni, infatti successivamente alla clone dobbiamo chiamare la clone degli oggetti interni, oppure a mano nel caso di array

Ovviamente bisogna scegliere in base al contesto e alla situazione se usare **shallow copy** (superficiale) oppure **deep copy** (profonda)

Static

Le variabili e i metodi definiti *static* sono associati alla classe e non all'oggetto.
Infatti si può definire una variabile statica all'interno di una classe per avere quella variabile condivisa in ogni istanza di quella classe (ad esempio un contatore di istanze).
Per poter usare una variabile statica di una classe non devo fare *nomeOggetto*.*var* ma *nomeClasse*.*var*.
La stessa cosa vale per i metodi statici, e ovviamente può agire (chiamare) solo variabili statiche della classe.
Le variabili e i metodi statici possono essere usati anche senza aver creato nessuna istanza di quella classe.



Il metodo $main()$ è statico perché serve come entry point quando si runna il programma, e quindi deve già esistere. Visto che il main è statico, è buona norma mettere la logica del programma dentro al costruttore della classe che contiene il main, cosi che si possano usare anche variabili non statiche della classe, altrimenti se si facesse tutto dentro al main non si potrebbero usare variabili non statiche.
I parametri di ingresso (ovvero i parametri del main, quindi un array di String), possono essere inseriti in Intellij dalla configurazione di Run.

Javadoc

Quando viene creato, un nuovo progetto, il codice generato contiene già lo scheletro di base del JavaDoc :

```
package throwaway2;
/**
 * Qui va messa una breve descrizione della classe
 * @author ronchet
 */
public class ThrowAway2 {
    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        // TODO code application logic here
    }
}
```

Si possono mettere le descrizioni del metodo e dopo la *@ param* la descrizione del parametro.
Si usano per documentare classi, metodi e variabili pubbliche

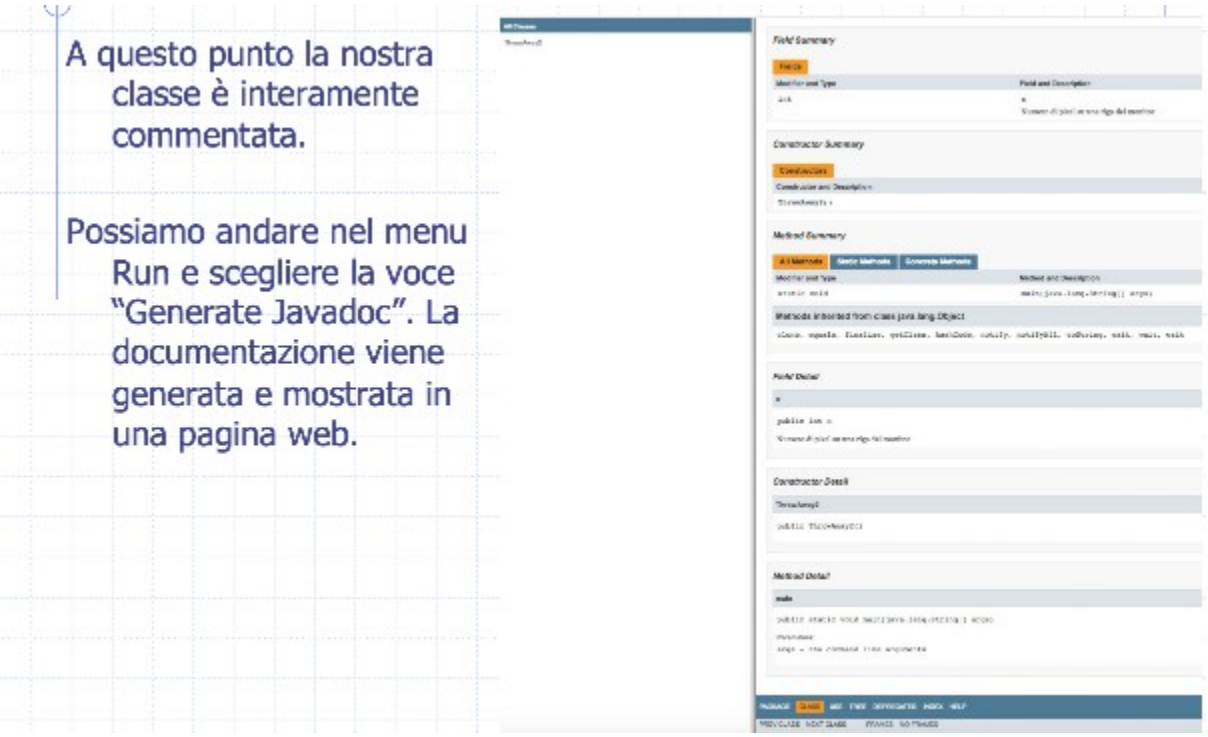
11

Documentazione di metodi

Se ora apriamo una riga prima della dichiarazione del metodo, digitiamo /** (apertura del commento) e diamo return, Netbeans prepara lo scheletro di documentazione del metodo stesso (in rosso).

A noi resta da scrivere la spiegazione di cosa fa il metodo, e il significato dei parametri (testo in verde)

```
/**
* Un metodo che non fa nulla
* @param k sceglie come fare l'azione richiesta:
* se k=0 non fa niente, se k!=0 non fa nulla.
*/
void f(int k) {
   //do nothing
}
```



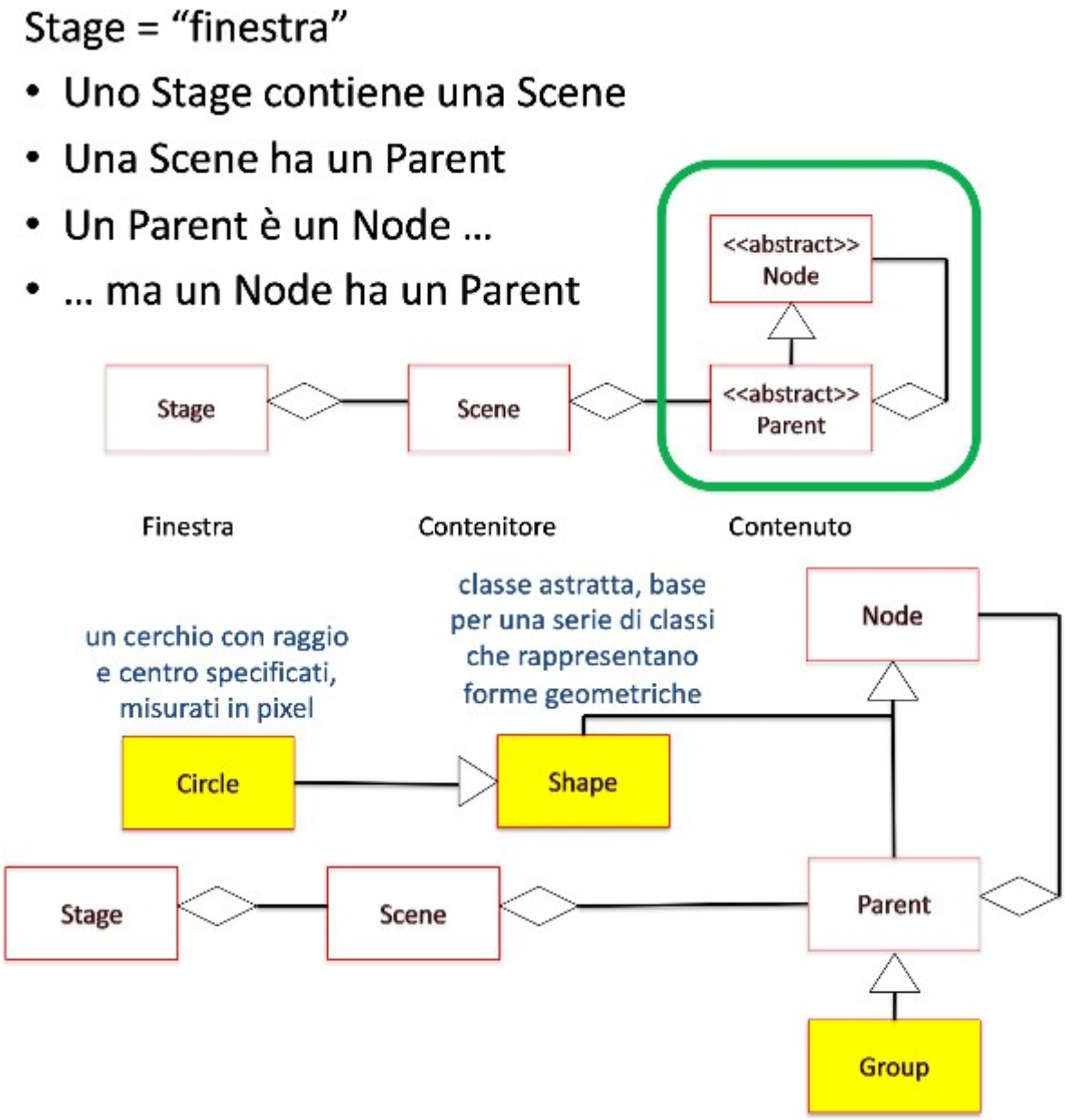
Java FX

Java funziona su ogni tipo di macchina perché ogni sistema operativo ha la sua JVM che traduce le classi compilate in bytecode apposta per quel sistema operativo.
Un problema ad esempio è che i vari sistemi operativi hanno più tasti del mouse, però la cosa in comune è che tutti ne abbiano almeno uno, quindi questa è l'assunzione da fare quando useremo il click del mouse.

Esistono varie librerie per la grafica di Java, di default c'è AWT, però noi useremo FX, quindi quando IntelliJ ci consiglierà di implementare una libreria, ci proporrà tutte quelle possibili, bisogna stare attenti a scegliere quella giusta (FX)

Il framework è la base da dove si parte nella programmazione moderna.
Nel framework ci sono delle classi, e il codice che noi scriviamo farà delle chiamate agli oggetti istanziati dentro al framework.
Uno di questi metodi è il metodo $launch(args)$, che prepara il framework e chiama il metodo $start$, di cui noi facciamo overriding.
Quindi il nostro codice dovrà essere scritto in $start$

Stage



A differenza di un’applicazione standard, una applicazione Java FX non interrompe l’esecuzione quando “finiscono” le istruzioni, bensì si mette in attesa di un’azione dell’utente. Ad esempio cliccando sulla x della finestra oppure il pulsante di interruzione direttamente da IntelliJ.

Un input senza grafica si fa così:

```
import java.util.Scanner;
...
Scanner scanner = new Scanner(System.in);
System.out.println(”dimmi qualcosa");
String inputString = scanner.nextLine();
    ...
System.out.println(inputString);
```

Un output con grafica si fa così:

```
Alert alert = new Alert(AlertType.INFORMATION);
alert.setTitle("Alert Title");
alert.setHeaderText("Alert header:");
alert.setContentText("Alert Text");
alert.showAndWait();
```



Un input con grafica si fa così:

```
TextInputDialog dialog = new TextInputDialog("Default
    answer");
dialog.setTitle("Dialog Title");
dialog.setHeaderText("Dialog header");
dialog.setContentText("Answer label:");
String s= dialog.showAndWait().get();
```

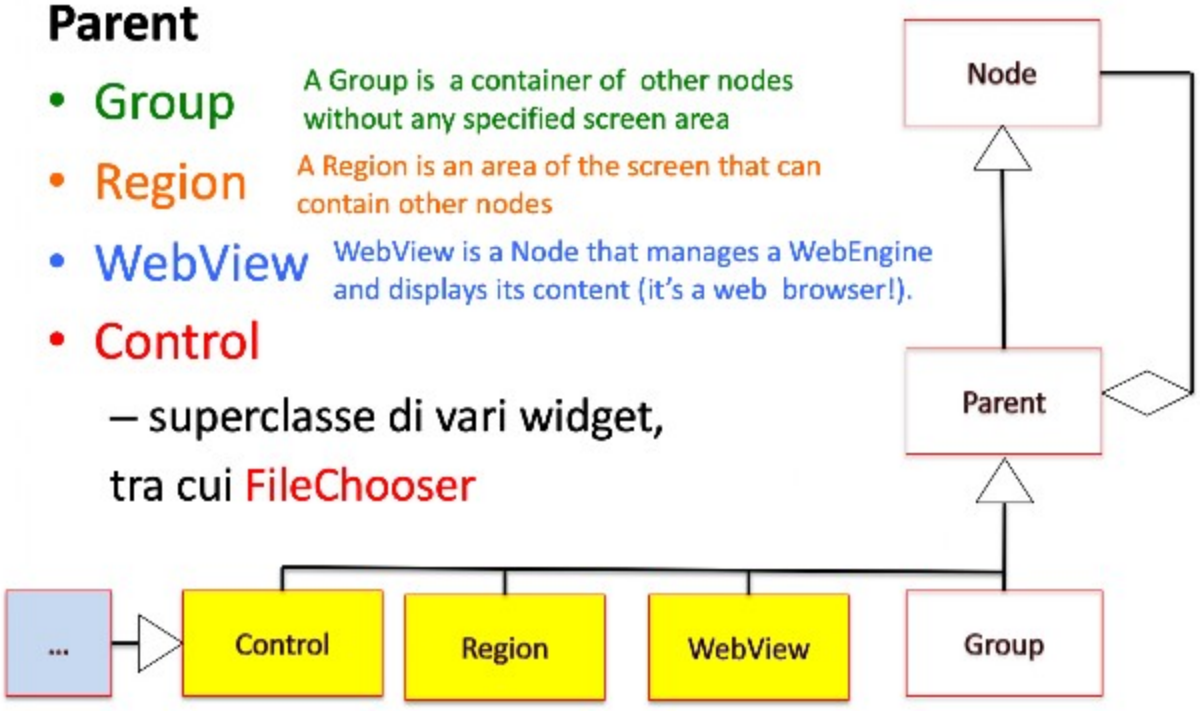
Non è proprio il modo giusto per farlo, ma per ora...



Parent hierarchy

Parent

- Group — A Group is a container of other nodes without any specified screen area
- Region — A Region is an area of the screen that can contain other nodes
- WebView — WebView is a Node that manages a WebEngine and displays its content (it's a web browser!).
- Control
  - – superclasse di vari widget, tra cui FileChooser



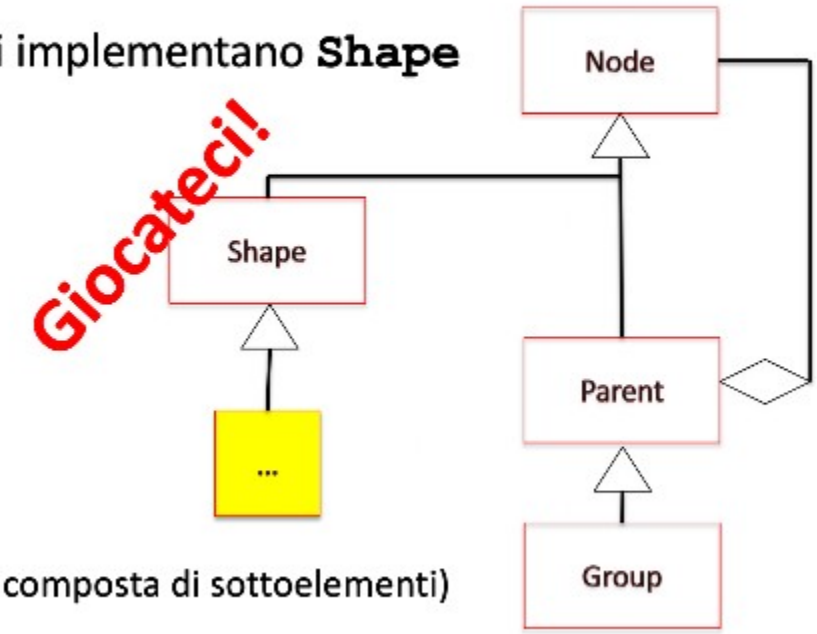
Un'altra sottoclasse di *Shape* è *Text* (per inserire testo nell'interfaccia grafica)

Gerarchia di Shape

Le seguenti classi implementano **`Shape`**

- **`Line`**
- **`Polyline`**
- **`Polygon`**
- **`Rectangle`**
- **`Arc`**
- **`Circle`**
- **`Ellipse`**
- **`QuadCurve`**
- **`CubicCurve`**
- **`Text`**
- **`SVGPath`** (linea composta di sottoelementi)

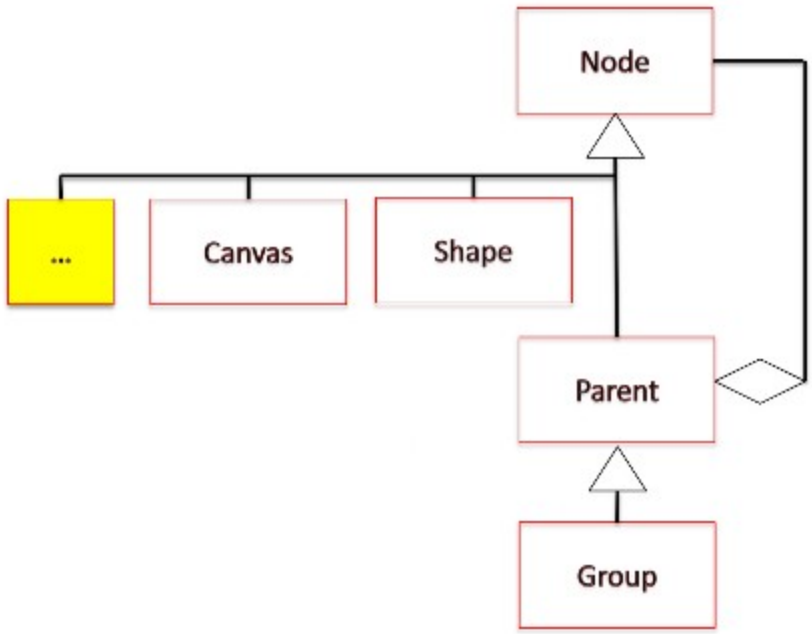
Giocateci!



Node hierarchy

Node

- Parent
- Shape
- Canvas



La differenza tra Canvas e Shape è che le cose “disegnate” sul Canvas (tela del pittore) non hanno un’identità, quindi non può essere modificato/eliminato, invece nel Shape posso farlo.

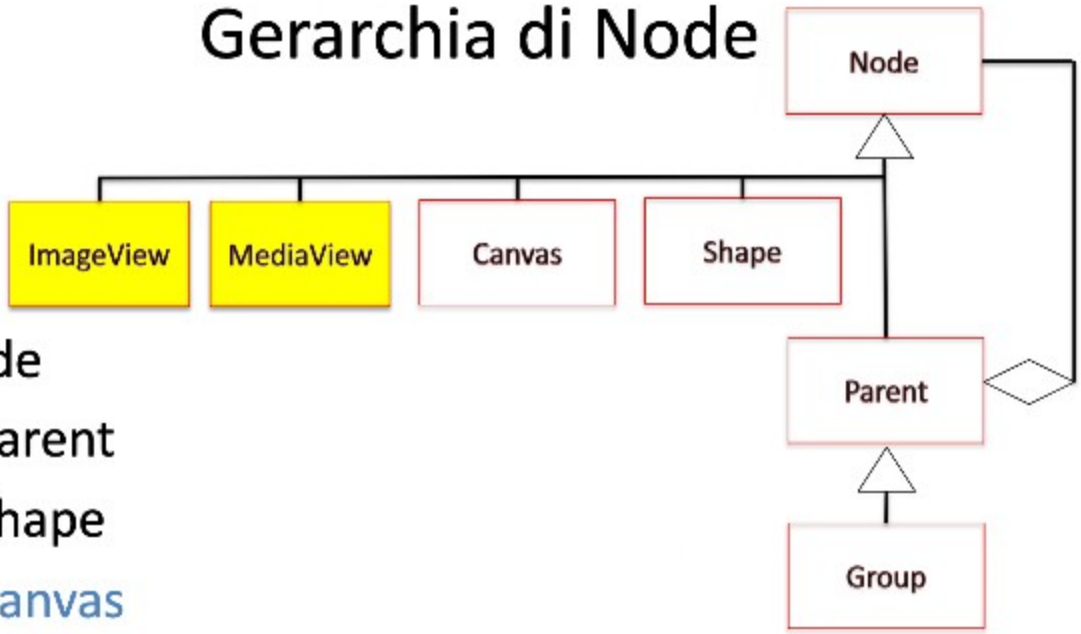
Canvas

Una "tela del pittore" con un metodo per ottenere il suo `GraphicContext` con varie primitive per disegnarci sopra:

- `fillArc()`
- `fillRect()`
- `drawImage()`
- ...

http://docs.oracle.com/javafx/2/canvas/jfxpub-canvas.htm

Gerarchia di Node

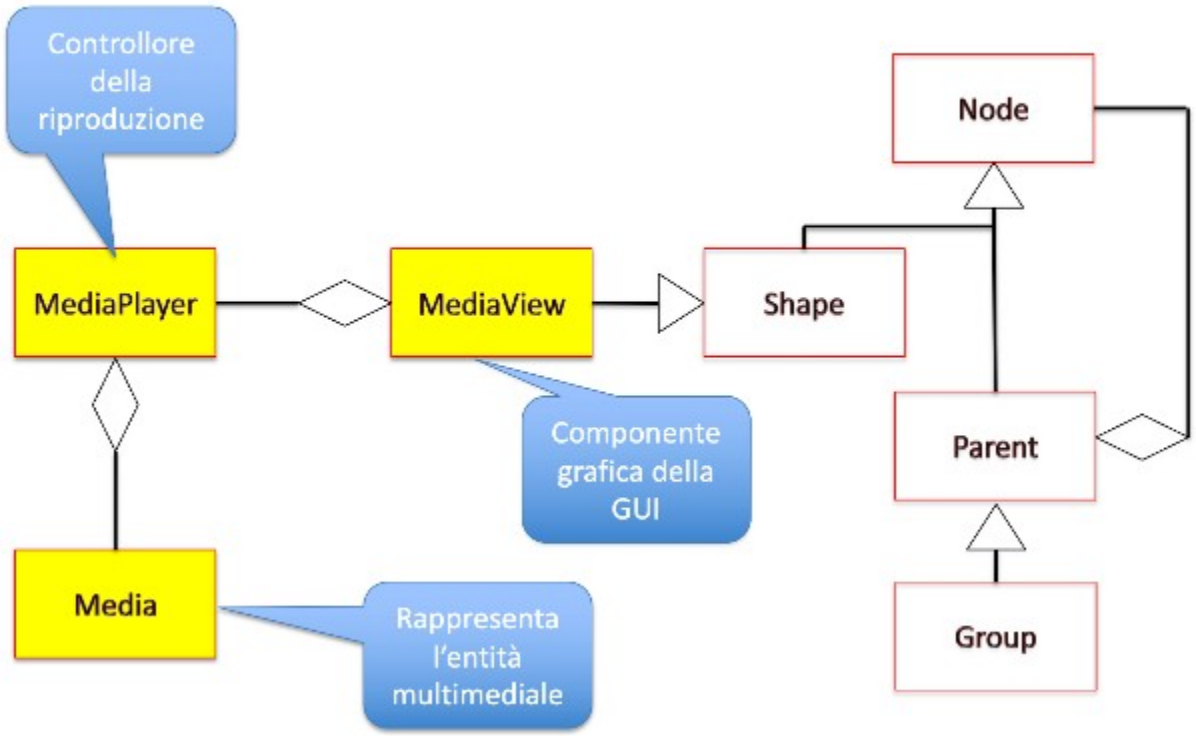


Node

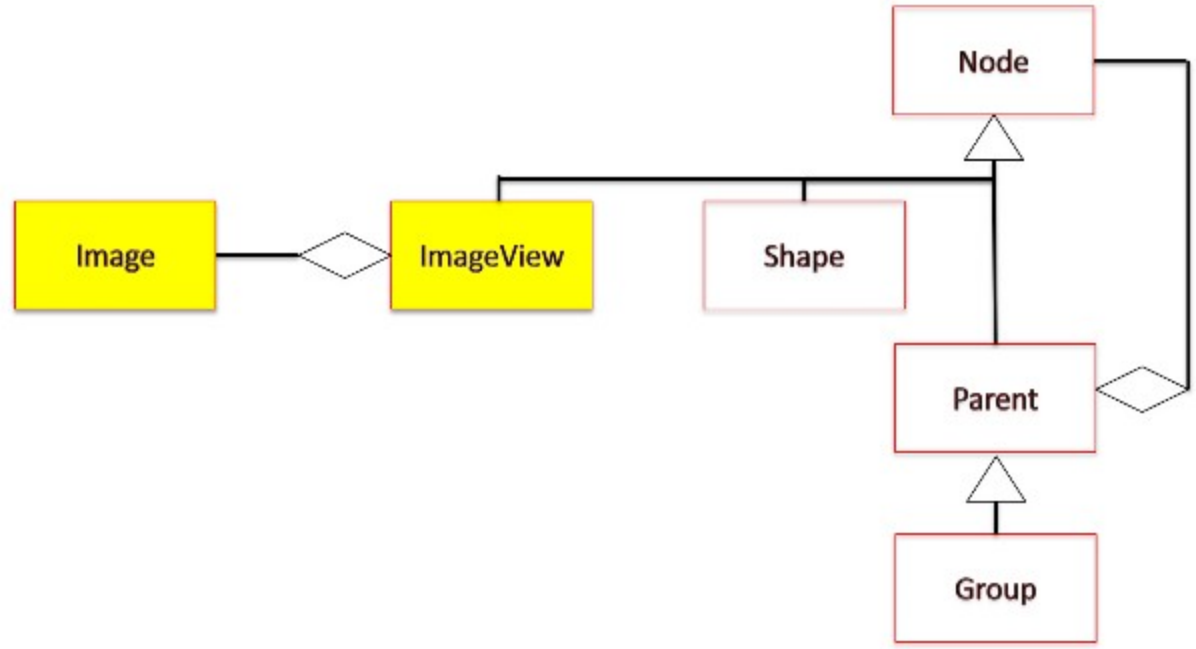
- Parent
- Shape
- Canvas
- ImageView
- MediaView

Il problema di avere video o audio è che hanno una durata e devono essere gestiti in qualche modo.

MediaView & Media



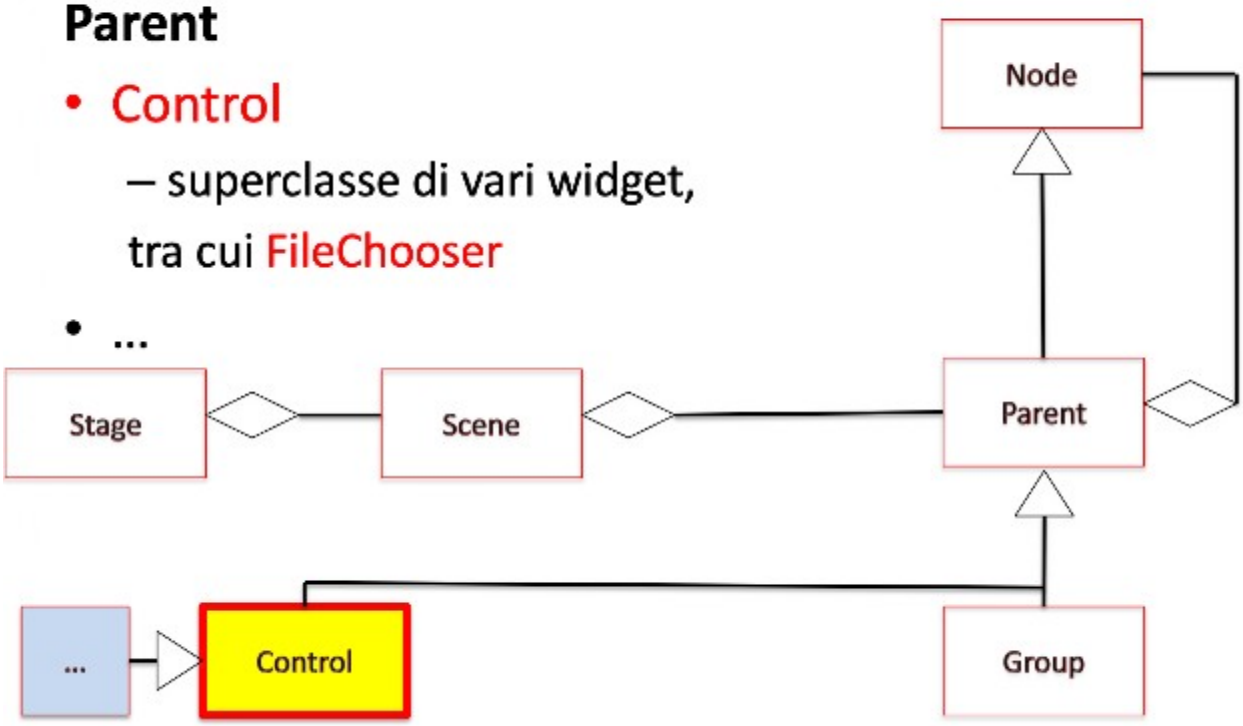
ImageView & Image



Gerarchia di Parent (parziale)

Parent

- Control
  - superclasse di vari widget, tra cui FileChooser
- ...



Finestre multiple: prima finestra

```
public class Finestre extends Application {
public void start(Stage stage) {
    Text t=new Text(50, 100, "The quick brown fox jumps over
       the lazy dog");
    t.setTextAlignment(TextAlignment.RIGHT);
    t.setWrappingWidth(50);
    t.setFill(Paint.valueOf("GREEN"));
    Group g = new Group(t);
    Scene scene = new Scene(g);
    stage.setTitle("Titolo
                della finestra 1");
    stage.setScene(scene);
    // set stage dimension
    stage.setWidth(399);
    stage.setHeight(399);
   // set stage position
    stage.setX(400);
    stage.setY(400);
    // make stage visible
    stage.show();
```



Finestre multiple: seconda finestra

```
        Text t2=new Text(0, 20, "Ping !\nPongPing !");
        t2.setTextAlignment(TextAlignment.LEFT);
        t2.setFill(Paint.valueOf("RED"));
        t2.setFont(new Font(20));
        Group g2 = new Group(t2);
        Scene scene2 = new Scene(g2);
        scene2.setFill(Paint.valueOf("YELLOW"));
        Stage stage2 = new Stage();
        stage2.setTitle("Titolo della finestra 2");
        stage2.setScene(scene2);
        stage2.sizeToScene();
        stage2.setX(100);
        stage2.setY(80);
        stage2.show();
    }
    public static void main(String[] args) {
           launch(args);
    }
}
```



Non è un buon modo di programmare prendere un template e cambiare solo alcune cose senza sapere cosa c'è dietro.

Eventi

Un framework è un insieme di codice già scritto che noi possiamo usare.

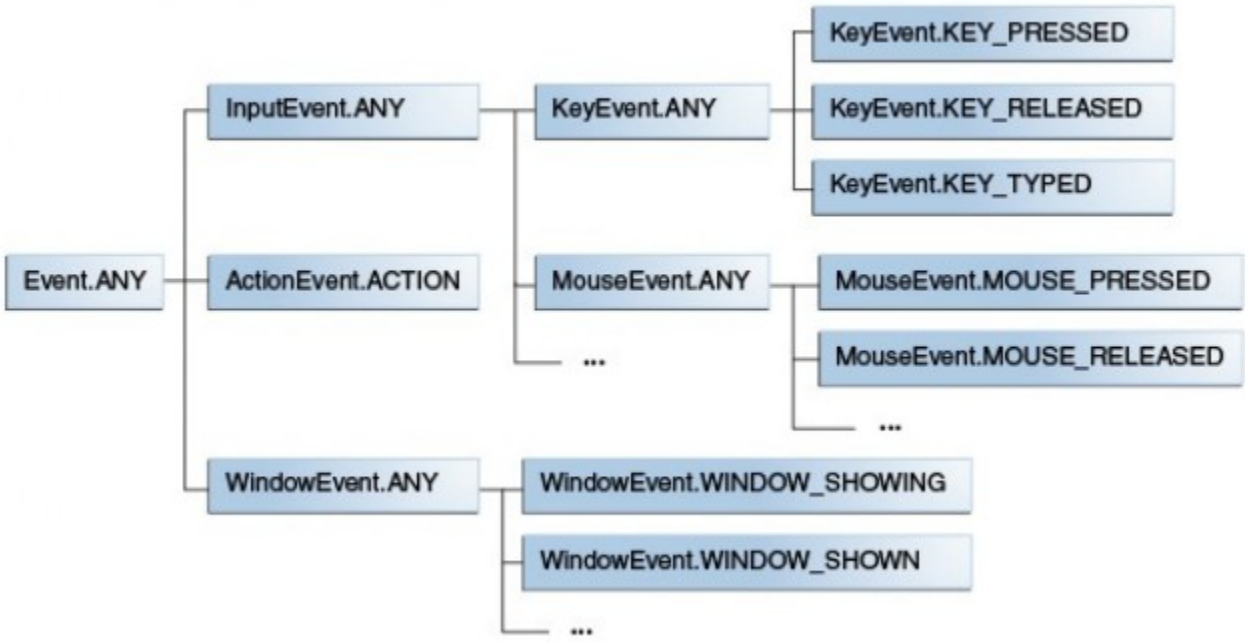
Alcuni eventi base

```
1 Ricevuto un evento di tipo
            INPUT_METHOD_TEXT_CHANGED
2 Ricevuto un evento di tipo MOUSE_ENTERED
3 Ricevuto un evento di tipo
            MOUSE_ENTERED_TARGET
4 Ricevuto un evento di tipo MOUSE_MOVED
...
12 Ricevuto un evento di tipo MOUSE_MOVED
13 Ricevuto un evento di tipo MOUSE_PRESSED
14 Ricevuto un evento di tipo ACTION
15 Ricevuto un evento di tipo MOUSE_RELEASED
16 Ricevuto un evento di tipo MOUSE_CLICKED
17 Ricevuto un evento di tipo MOUSE_MOVED
```

```
public class Event0 extends Application {
      public void start(Stage stage) {
        Button btn = new Button();
        btn.setText("Click me");
        Listener a=new Listener();
        btn.addEventHandler(Event.ANY, a);
        Group root = new Group(btn);
        Scene scene = new Scene(root, 300, 250);
         stage.setScene(scene);
        stage.sizeToScene();
        stage.show();     }
    public static void main(String[] args){
       Application.launch(args); }
}
class Listener implements EventHandler {
    int counter=0;
    public void handle(Event t) {
     System.out.println(++counter+" Ricevuto un evento di tipo "
        +t.getEventType()); } }
```



JavaFX definisce una gerarchia di eventi predefinita e ogni oggetto *Eventi* raggruppa uno o più sotto eventi, i quali sono definiti da **costanti**.



Il codice applicativo può reagire a questi eventi specificando uno o più *Listener* o *Controller*, che devono implementare l'interfaccia *EventHandler* e sovrascrivere il metodo *handle*.

Aggiunta – rimozione di un ascoltatore

```
EventHandler lstn=new Listener()

btn.addEventHandler(
    ActionEvent.ACTION,lstn);

btn.removeEventHandler(
    ActionEvent.ACTION, lstn);
```

È possibile avere dei Listener integrati nella classe, oppure direttamente la classe stessa che contiene il bottone può implementare *EventHandler*.

1. **Self** (la classe principale implementa *EventHandler*)
   In questo caso devo scrivere nella classe principale il metodo *handle* e dovrò passare ai

bottoni *this*

Listener “integrato”



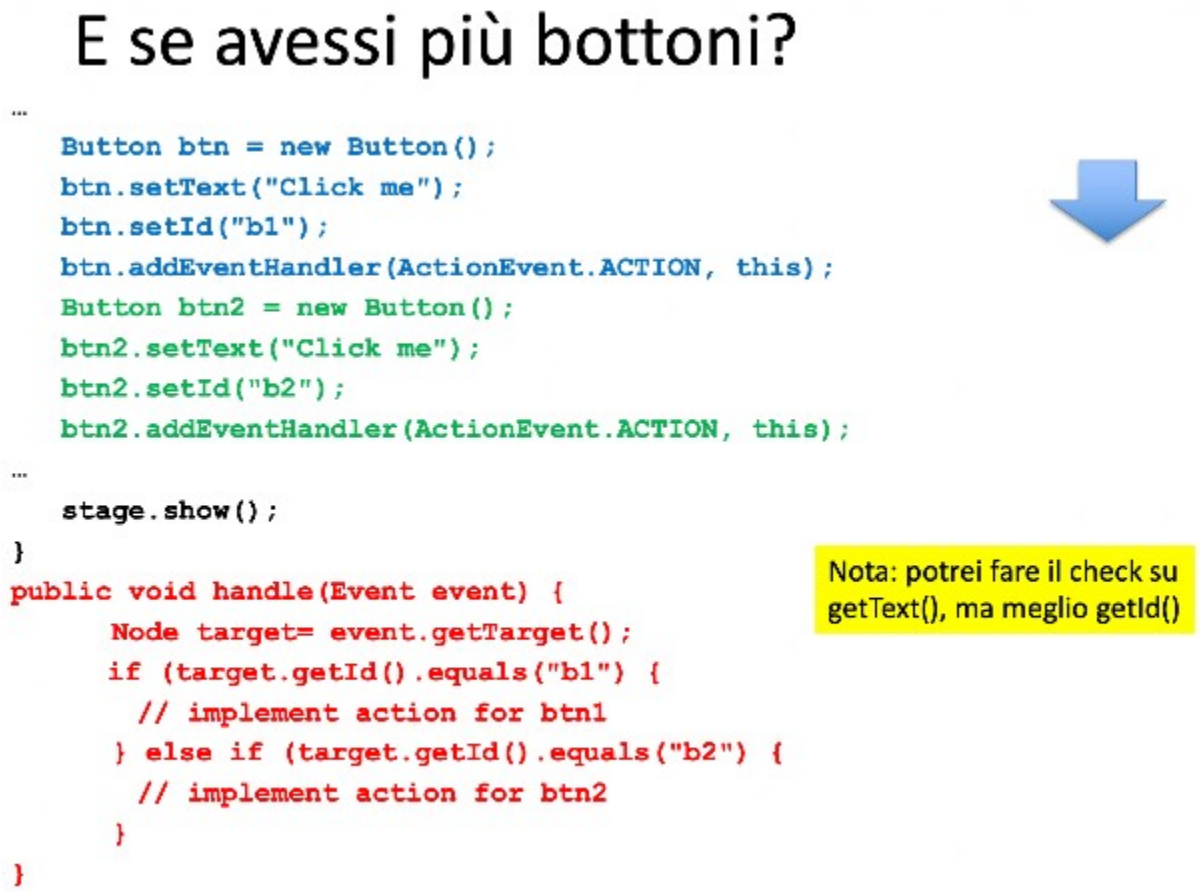

```
public class AppWithEvents extends Application
             implements EventHandler {
    Text text = null;
    int counter = 0;
    public void start(Stage stage) {
      text = new Text(10,50,"Non hai mai cliccato ");
      Button btn = new Button();
      btn.setText("Click me");
      btn.addEventHandler(ActionEvent.ACTION, this);
      Group root = new Group(btn);
      root.getChildren().add(text);
      Scene scene = new Scene(root);
      stage.setScene(scene);
      stage.show();
    }
    public void handle(Event t) {
      updateText(++counter);
    }
```

```
(Compiler) warning:
uses unchecked or unsafe operations
```

```
    public void updateText(int n){
      text.setText("Hai cliccato "
                   +n+" volte");
    }
    public static void main(
         String[] args) {
      launch(args);
    }
} // end of AppWithEvents
```

Nel caso in cui avessi più bottoni, e stessi usando direttamente l’handle della classe dove ho i bottoni, dovrei andare a controllare dentro *handle* quale bottone è stato

cliccato, tramite $event.getTarget().getId()$.



2. **Classi esterne**
   In alternativa si possono creare delle classi esterne che estendono *EventHandler*, ma in questo caso dovrei passare tutti gli oggetti del *Main* che dovranno essere modificati al click, che non sono visibili dalla classe del Listener (si fa prima a passare direttamente

tutta la classe al momento della creazione del Listener)

Listener esterno




```
public class AppWithEvents extends Application {
    Text text = null;
    public void start(Stage stage) {
      text = new Text(10,50,"Non hai mai cliccato ");
      Button btn = new Button();
      btn.setText("Click me");
      Listener a = new Listener(this);
      btn.addEventHandler(ActionEvent.ACTION, a);
      Group root = new Group(btn);
      root.getChildren().add(text);
      Scene scene = new Scene(root);
      stage.setScene(scene);
      stage.show();
    }
    public void updateText(int n){
      text.setText("Hai cliccato "
                   + n +" volte");
    }
    public static void main(String[] args) {
      launch(args);
    }}
```

```
class Listener
      implements EventHandler {
  AppWithEvents awe = null;
  int counter = 0;
  Listener(AppWithEvents a){
    awe = a;
  }
  public void handle(Event t) {
    awe.updateText(++counter);
  }
}
```

Dichiarato fuori dalla classe principale

3. **Classi interne**
   Altrimenti posso creare una classe interna alla classe del bottone e creare lì il listener, così facendo ho già la visibilità su tutte le componenti della classe superiore e quindi non

devo preoccuparmi di passare gli oggetti.

Listener interno – 2a




```
public class AppWithEvents extends Application {
    Text text = null;
    public void start(Stage stage) {
      text = new Text(10,50,"Non hai mai cliccato ");
      Button btn = new Button();
      btn.setText("Click me");
      Listener a = new Listener();
      btn.addEventHandler(ActionEvent.ACTION, a);
      Group root = new Group(btn);
      root.getChildren().add(text);
      Scene scene = new Scene(root);
      stage.setScene(scene);
      stage.show();
    }
    public void updateText(int n){
       text.setText("Hai cliccato "
          +n+" volte");
    }
    //continua nel riquadro a dx
```

```
public static void main(
      String[] args) {
   launch(args);
 }
 class Listener
     implements EventHandler{
     int counter =  0;
     public void handle(Event t) {
       updateText(++counter);
     }
   } // end of Listener
} // end of AppWithEvents
```

4. **Classi interne anonime**
   Posso anche creare un Listener interno anonimo, cioè senza definire una classe interna ma semplicemente passare all' *addEventHandler* del bottone un oggetto creato al momento: $new\ \ EventHandler()\{\ldots\ \ public\ \ void\ \ handle(\ldots)\ldots\};$
   Questo si fa quando il Listener viene usato un' unica volta e solo per uno specifico

contesto

Listener interno anonimo




```
public class AppWithEvents extends Application {
    Text text = null;
    public void start(Stage stage) {
      text = new Text(10,50,"Non hai mai cliccato ");
      Button btn = new Button();
      btn.setText("Click me");
      btn.addEventHandler(ActionEvent.ACTION,
       new EventHandler(){
           int counter=0;
           public void handle(Event t) {
             updateText(++counter);
           }
       }
      );
      Group root = new Group(btn);
      root.getChildren().add(text);
      Scene scene = new Scene(root);
      stage.setScene(scene);
      stage.show();
    }
    public void updateText(int n){
      text.setText("Hai cliccato "
                   +n+" volte");
    }
    public static void main(
         String[] args) {
      launch(args);
    }
} // end of AppWithEvents
```

Bisogna stare attenti quando si usano classi interne anonime perchè ci possono essere problemi per quanto riguarda variabili create in un livello dello stack che verrà poi eliminato.
Se non facciamo “operazioni” su una variabile, quella verrà interpretata effettivamente come *final*, e quindi avrà una vita più lunga di quella della funzione dove è dichiarata.
Quindi da una classe interna anonima, posso accedere alle variabili locali della funzione esterna alla classe interna, quelle variabili devono essere dichiarate final (oppure effectively final, ovvero interpretate come final perchè non vengono modificate).
Per poter modificare la variabile devo metterla nello heap, quindi ad esempio posso metterla come variabile di istanza.

Gli handler hanno come parametro $<typeEvent>$ in cui possiamo specificare che eventi vengono gestiti.
Al posto di usare $btn.addEventHandler(ActionEvent.ACTION, this)$ posso usare anche $btn.setOnAction(this)$ dove *this* è la classe in cui ho fatto *implements* $EventHandler$.
Di solito si usa questa sintassi quando usiamo una classe interna anonima, e sono definiti per gli eventi più comuni.

Lambda expressions

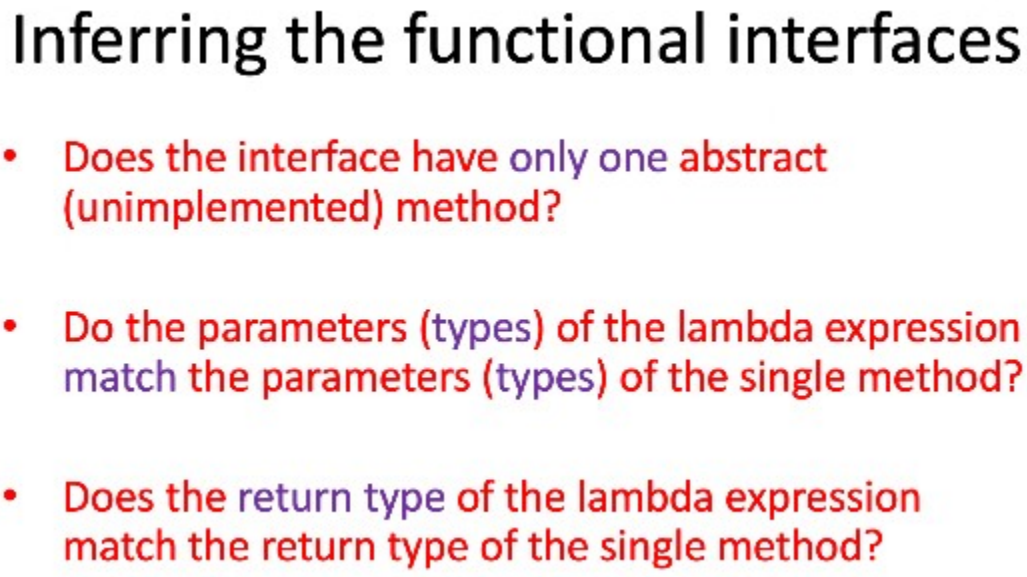
Si può sostituire una classe interna anonima con una lambda expression, ovvero scrivere la classe interna anonima con dei dati in meno, come il nome della classe e il nome del metodo astratto che si implementa.

```
c.setOnMouseEntered(new EventHandler<MouseEvent>() {
  public void handle(MouseEvent event) {
      System.out.print("Entered");
        c.setFill(Color.RED);
});

c.setOnMouseEntered((MouseEvent event) -> {
      System.out.print("Entered");
      c.setFill(Color.RED);
  });
```

Per poter eseguire questa operazione ci sono delle condizioni, alcune domande da farsi per capire se la lambda expression è stata usata correttamente:

- L'interfaccia deve avere un solo metodo da implementare (in questo caso handle)
- Il parametro della lambda expression deve essere **unico** e deve coincidere con il parametro di questo metodo astratto dell'interfaccia
- Il tipo di ritorno deve coincidere con quella della lambda expression (in questo caso void)



Si può anche omettere il tipo del parametro visto che sono sottintesi essendo il metodo da chiamare unico.

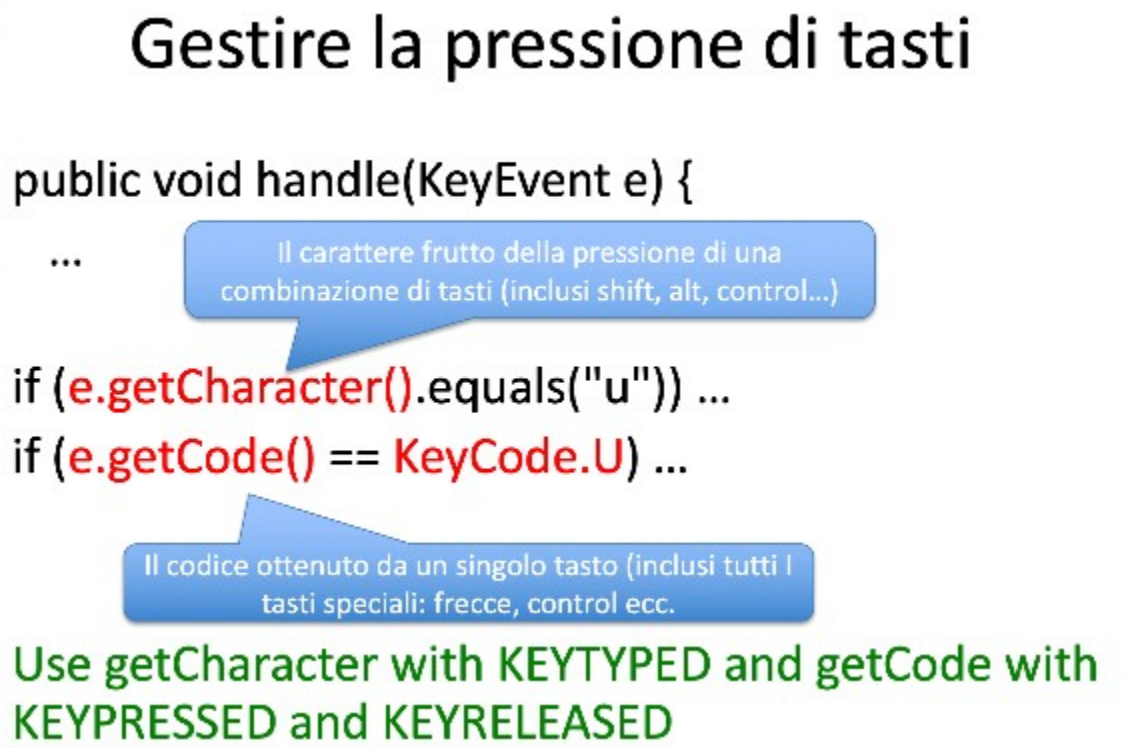
Gestire la pressione di tasti della tastiera

Invece di usare $ActionEvent$ possiamo usare un'altra di tipologia di evento, relativo alla pressione di tasti sulla tastiera: $KeyEvent$

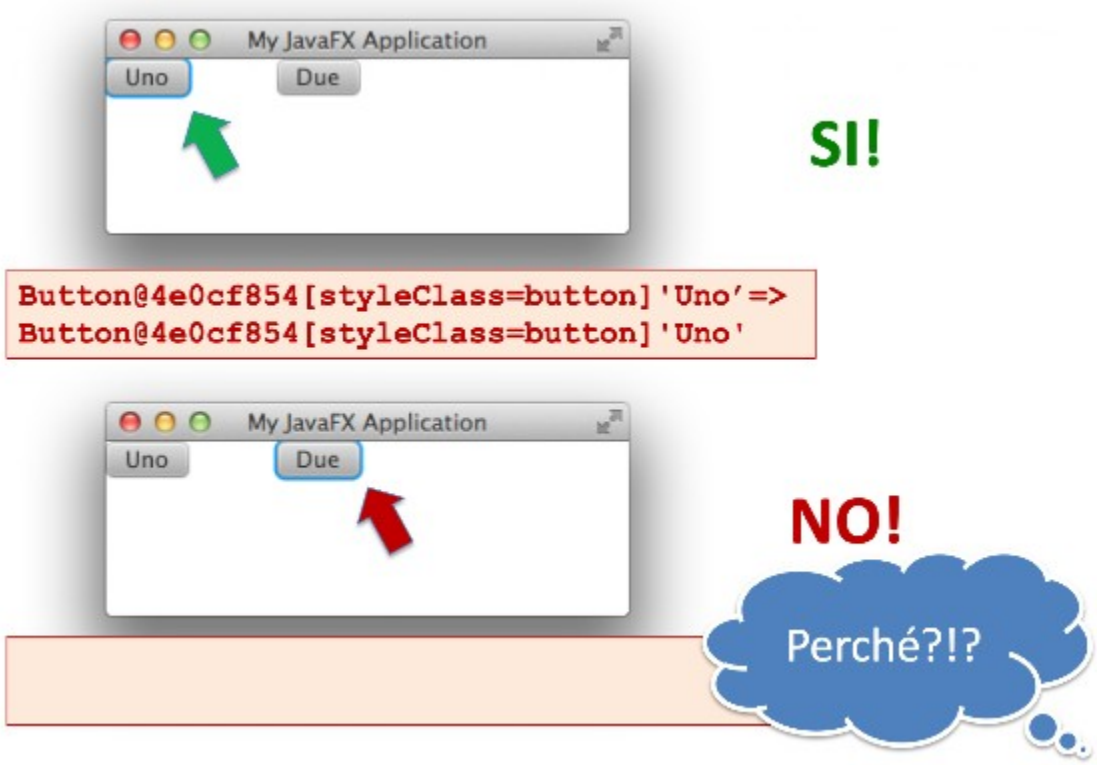
Per poter vedere che tasto è stato cliccato, nell'handle dichiarato come $public\ \ void\ \ handle\ \ (KeyEvent\ \ e)$ possiamo usare $e.getCharacter().equals...$ e possiamo avere la combinazione di tasti cliccata (quindi ad esempio "SHIFT" + "u").

Se volessimo vedere qualsiasi tasto cliccato, compresi i tasti "modificatori", usiamo $e.getCode()$ e posso confrontarlo con $KeyCode.U$ ad esempio

Di conseguenze gli eventi di tipo KEYPRESSED e KEYRELEASED verranno usati con $getCode$, invece gli eventi di tipo KEYTYPED verranno usati con $getCharacter$



Per poter gestire un evento posso far in modo che il click della tastiera generi un evento su un bottone, così da fare in modo che il click della tastiera o il click sul bottone faccia la stessa cosa, per chiamare l'evento sul bottone uso $b1.fireEvent(new\ \ ActionEvent());$



Il problema è che se aggiungo un altro bottone, nel caso in cui sia “evidenziato” (ovvero abbia il focus) l’altro bottone, se clicco “u” (che è l’evento che fa l’action sull’altro bottone) non succede niente.
Ci sono delle regole per decidere il “target” di un evento:

- Key events
  Il target sarà il nodo che ha il focus
- Mouse events
  Il target sarà il nodo che ha il puntatore del mouse sopra di lui

Questo perché l’evento di KeyEvent l’abbiamo associato al bottone Uno, quindi viene chiamato solo quando il bottone è “selezionato” (concetto di focus).

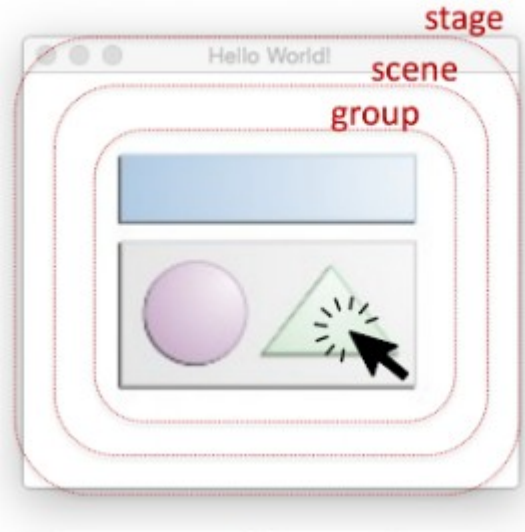
```
// dentro start()…
EventHandler<KeyEvent> keyEventHandler =
  new EventHandler<KeyEvent>(){
  public void handle(KeyEvent keyEvent) {
    if (keyEvent.getCharacter().equals("u")) {
      b1.fireEvent(new ActionEvent());
      System.out.println(keyEvent.getSource() +
                         " =>"+keyEvent.getTarget());
    }
  }
};
b1.addEventHandler(KeyEvent.KEY_TYPED,keyEventHandler);
```

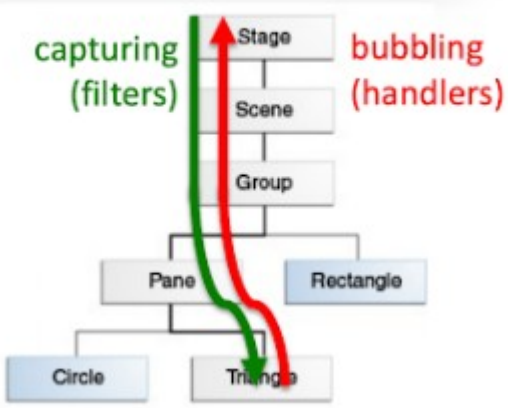
siamo noi a generare un **ActionEvent!**

A volte può essere utile far gestire un evento al contenitore e non al contenuto.
Regola base: **tutti gli eventi partono dallo stage, arrivano al target, e tornano allo stage**.
Seguendo questa logica è molto semplice intercettare gli eventi dove interessa a noi.

- ***event capturing***: stage ➔ target
  - eventi intercettati mediante ***filter***
- ***event bubbling***: target ➔ stage
  - eventi intercettati mediante ***handler***

Gli event e i filter sono uguali, cambia solo il modo con cui li "attacchiamo" al nostro node, quindi o con *addEventHandler* oppure con *addEventFilter*





Quindi in base a se uso *filter* o *handler* posso decidere in che fase intercettare l'evento (in base alle nostre esigenze)
La sequenza di componenti tra stage e target si chiama **event dispatch chain**

Con la keyword $t.consume()$, dove t è l'evento che viene passato come parametro su *handle*, per interrompere la propagazione dell'evento lungo la event dispatch chain (questo tipo di handler lo chiamiamo **cutter**).
Per ottenere il nome di una classe di un oggetto posso usare
$source.getClass().getSimpleName().toUpperCase()$

Nel caso di KeyEvent è meglio attaccare l'evento allo stage, così non dobbiamo preoccuparci del focus. Se vogliamo gestire altri eventi da tastiera basta fare degli if per capire quale tasto è stato cliccato.

```
// dentro start()…
EventHandler<KeyEvent> keyEventHandler =
  new EventHandler<KeyEvent>(){
  public void handle(KeyEvent keyEvent) {
     System.out.println(keyEvent.getSource() +
                        " =>"+keyEvent.getTarget());
    switch (keyEvent.getCharacter()){
       case "u":
       case "U":
         b1.fireEvent(new ActionEvent()); b1.requestFocus();
         break;
       case "d":
       case "D":
         b2.fireEvent(new ActionEvent()); b2.requestFocus();
         break;
  }
};
stage.addEventHandler(KeyEvent.KEY_PRESSED,keyEventHandler);
```

Possiamo anche fare che ogni volta che premiamo un tasto sulla tastiera, il focus viene spostato sul bottone a lui associato con $b1.requestFocus()$, così da migliorare l'impatto grafico e far capire all'utente cosa succede.

Posizionamento componenti

Metodi per il posizionamento **assoluto** di un *node* non vengono in genere forniti, sebbene sono presenti in alcune sottoclassi. Vengono forniti invece *setLayout* e *setTranslate*
Per fortuna il posizionamento (il quale è un problema considerando le varie risoluzioni diverse che hanno gli schermi) viene gestito per la maggior parte da JavaFX.
In FX la responsabilità della posizione dei componenti viene attribuita ad un tipo particolare di contenitori (o Parent): i Pane

Posizionamento *automatico* mediante *layout*

L'idea è di semplificare il compito del programmatore definendo ***contenitori*** di oggetti che vengono posizionati secondo regole prestabilite.

Il contenitore può ignorare i desideri della componente (espressi da setLayoutX)

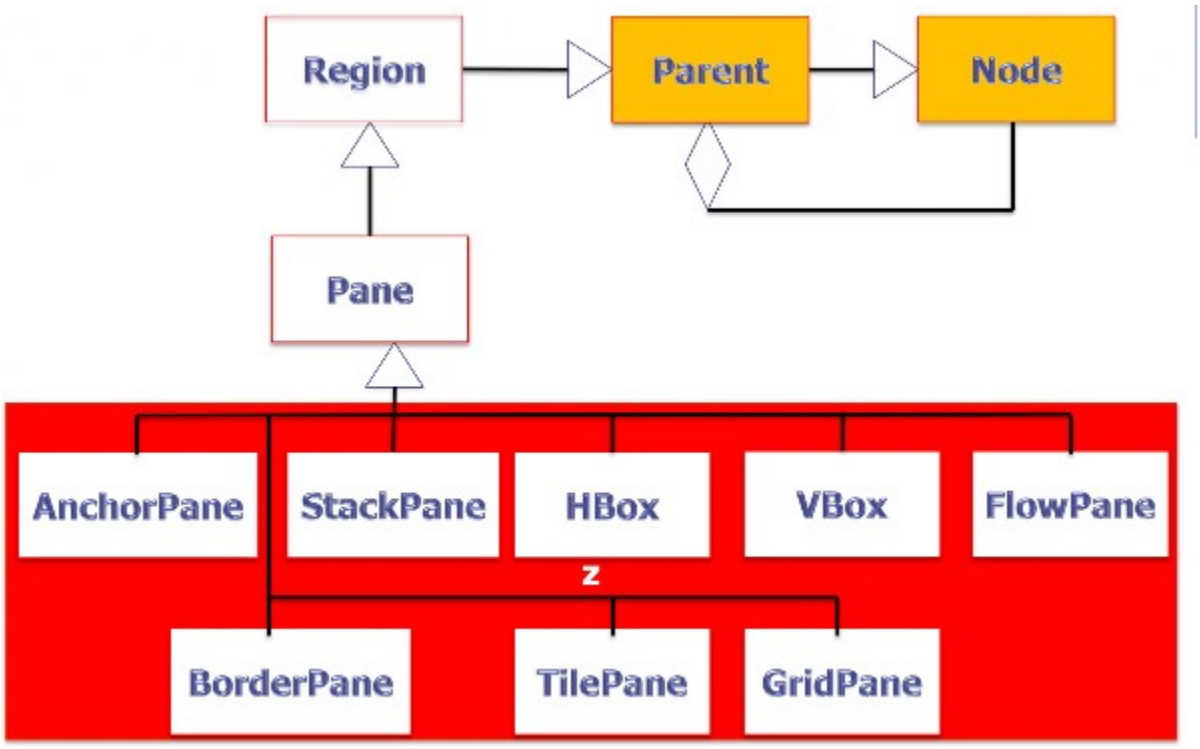
La disposizione delle componenti è ***"liquida"***

Per poter aggiungere un **nodo** ad un **contenitore** bisogna "lavorare" sulla sua lista dei **children**

Aggiunta e rimozione di elementi vengono effettuati dalla lista dei figli (children), con i metodi

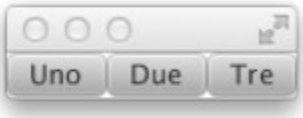
add(Node x), addAll(Collection<Node> c),
remove(Node x), , removeAll(Collection<Node> c)

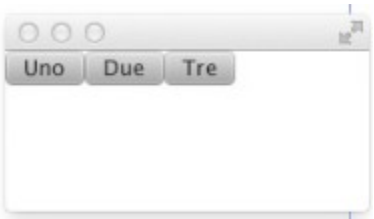
```
Polygon triangolo= new Polygon();
triangolo.getPoints().addAll(new Double[]{
   0.0, 0.0,20.0, 10.0, 10.0, 20.0 });
Circle cerchio=new Circle(10,10,10);
layout.getChildren().addAll(cerchio,triangolo);
…
layout.getChildren().remove(cerchio);
```



- HBOX
  Allinea i children in orizzontale
- VBOX
  Allinea i children in verticale
- FlowPane
  Allinea i children orizzontalmente fino a che c'è spazio, poi va a capo
- StackPane
  "Impila" i children uno sopra l'altro, ad esempio se si vuole mettere uno sfondo ad una scritta
- AnchorPane
  Per ancorare gli elementi ad una zona, definendone la distanza dai bordi ad esempio
- TilePane
  Organizza i children in una griglia di elementi di eguale dimensione
- GridPane
  Organizza i children in una griglia senza dimensione prefissata
- BorderPane
  Organizza gli elementi in zone (top,bottom,center etc)
- Pane
  Posizionamento assoluto (da evitare)

HBox

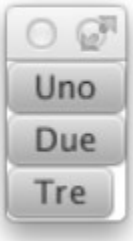





```
public class Layout1 extends Application {
    public void start(Stage stage) {
        Pane layout = new HBox();
        layout.getChildren().add(new Button("Uno"));
        layout.getChildren().add(new Button("Due"));
        layout.getChildren().add(new Button("Tre"));
        Group root = new Group(layout);
        Scene scene = new Scene(root);
        stage.setScene(scene);
        stage.show();
    }
    …
}
```

Allinea in orizzontale

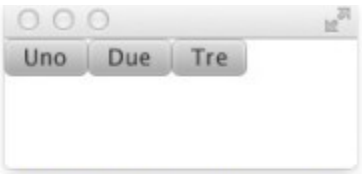
VBox




```
public class Layout1 extends Application {
    public void start(Stage stage) {
        Pane layout = new VBox();
        layout.getChildren().add(new Button("Uno"));
        layout.getChildren().add(new Button("Due"));
        layout.getChildren().add(new Button("Tre"));
        Scene scene = new Scene(layout);
        stage.setScene(scene);
        stage.show();
    }
...
}
```

Allinea in verticale

FlowPane






```
public class Layout1 extends Application {
    public void start(Stage stage) {
          FlowPane layout = new FlowPane();
        layout.setPrefWrapLength(100);
        layout.getChildren().add(new Button("Uno"));
        layout.getChildren().add(new Button("Due"));
        layout.getChildren().add(new Button("Tre"));
        Scene scene=new Scene(layout)
        stage.setScene(scene);
        stage.show();
    }
}
```

Organizza gli elementi in una sequenza continua, che va «a capo» a una distanza configurabile

StackPane

```
public class Layout1 extends Application {
    public void start(Stage stage) {
        StackPane layout = new StackPane();
        layout.getChildren().add(new Button("Uno"));
        layout.getChildren().add(new Button("Due"));
        layout.getChildren().add(new Button("Tre"));
        Scene scene = new Scene(layout);
        stage.setScene(scene);
        stage.show();
    }
...
}
```





Impila gli elementi

StackPane

```
public class Layout1 extends Application {
    public void start(Stage stage) {
        StackPane stack = new StackPane();
        Circle helpIcon = new Circle(15, 15, 15);
        helpIcon.setFill(Color.YELLOW);
        helpIcon.setStroke(Color.GREEN);
        Text helpText = new Text("?");
        helpText.setFont(Font.font("Verdana", FontWeight.BOLD, 18));
        helpText.setFill(Color.WHITE);
        helpText.setStroke(Color.RED);
        stack.getChildren().addAll(helpIcon, helpText);
        stack.setAlignment(Pos.CENTER);
        Scene scene = new Scene(stack);
        stage.setTitle("My JavaFX Application");
        stage.setScene(scene);
        stage.show();
    }
...
}
```

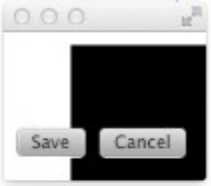




StackPane è utile per creare «overlay» di elementi

AnchorPane

```
public void start(Stage stage) {
    AnchorPane layout = new AnchorPane();
    Button buttonSave = new Button("Save");
    Button buttonCancel = new Button("Cancel");
    HBox hb = new HBox();
    hb.setPadding(new Insets(0, 10, 10, 10));
    hb.setSpacing(10);
    hb.getChildren().addAll(buttonSave, buttonCancel);
    Rectangle r=new Rectangle(100,100);
    layout.getChildren().addAll(r,hb);
    layout.setBottomAnchor(hb, 8.0);
    layout.setRightAnchor(hb, 5.0);
    layout.setTopAnchor(r, 10.0);
    layout.setLeftAnchor(r, 50.0);
    Scene scene = new Scene(layout);
    stage.setScene(scene);
    stage.show();
}
```



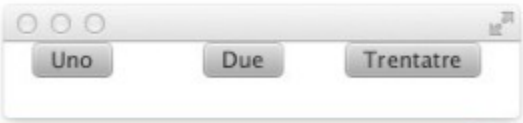


Permette di «ancorare» elementi a una zona

TilePane

```
public class Layout1 extends Application {
    public void start(Stage stage) {
        TilePane layout = new TilePane();
        layout.setVgap(10);
        layout.setHgap(20);
        layout.setPrefColumns(2);
        layout.getChildren().add(new Button("Uno"));
        layout.getChildren().add(new Button("Due"));
        layout.getChildren().add(new Button("Trentatre"));
              Scene scene = new Scene(layout);
        stage.setScene(scene);
        stage.show();
    }
…}
```



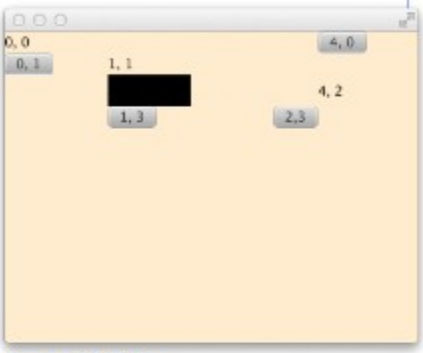




Organizza gli elementi in una griglia di celle di eguale dimensione

GridPane

```
public void start(Stage stage) {
  GridPane layout = new GridPane(); int width=100; int height=40;
  Scene scene = new Scene(layout, width, height, Color.BLANCHEDALMOND);
  layout.add(new Text("0, 0"), 0, 0);
  layout.add(new Button("0, 1"), 0, 1);
  layout.add(new Text("1, 1"), 1, 1);
  Rectangle r = new Rectangle(80,30);
  layout.add(r, 1, 2);
  layout.add(new Button("1, 3"), 1, 3);
  layout.add(new Button("2,3"), 2, 3);
  layout.add(new Button("4, 0"), 4, 0);
  layout.add(new Text("4, 2"), 4, 2);
  ColumnConstraints column1 = new ColumnConstraints(100);
  ColumnConstraints column2 = new ColumnConstraints();
  column2.setPercentWidth(40);
  column2.setHgrow(Priority.ALWAYS);
  layout.getColumnConstraints().addAll(column1, column2);
  stage.setScene(scene);
  stage.show();
}
```



vedi documentazione!

Organizza gli elementi in una griglia di cui non è necessario dare dimensione prefissata

Identificazione dell'elemento (i,j) in un GridPane

```
/**
 * implementazione generale del metodo per trovare quale elementi si trovi
 * in posizione i,j in un GridPane.
 * @param dp il GridPane in cui cercare
 * @param i riga
 * @param j colonna
 * @return l'elemento trovato
 */
Node getElementAt(GridPane gp, int i, int j) {
    for (Node x :gp. getChildren()) {
        if ((GridPane.getRowIndex(x) == i) &&
                (GridPane.getColumnIndex(x) == j)) {
            return x;
        }
    }
    return null;
}
```

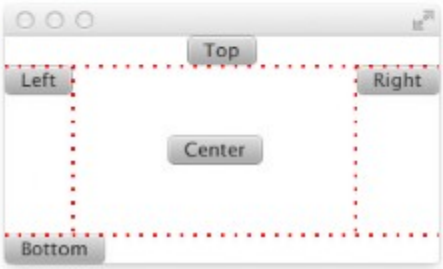
GridPane ha molti metodi interessanti e utili, ma ne manca uno che restituisca l'oggetto che si trova in posizione i,j nella matrice. Possiamo supplire così:

BorderPane

```
public class Layout1 extends Application {
  public void start(Stage stage) {
    BorderPane layout=new BorderPane();
    Button top = new Button("Top");
    BorderPane.setAlignment(top, Pos.TOP_CENTER);
    layout.setTop(top);
    layout.setBottom(new Button("Bottom"));
    layout.setLeft(new Button("Left"));
    layout.setRight(new Button("Right"));
    layout.setCenter(new Button("Center"));
    Scene scene = new Scene(layout);
       stage.setScene(scene);
    stage.show();
  }
...
}
```



Organizza gli elementi in «zone»



Combinazioni di layout



Container classes that automate common layout models

The HBox class arranges its content nodes horizontally in a single row.

The VBox class arranges its content nodes vertically in a single column.

The StackPane class places its content nodes in a back-to-front single stack.

The TilePane class places its content nodes in uniformly sized layout cells or tiles

The FlowPane class arranges its content nodes in either a horizontal or vertical "flow", wrapping at the specified width (for horizontal) or height (for vertical) boundaries.

The BorderPane class lays out its content nodes in the top, bottom, right, left, or center region.

The AnchorPane class enables developers to create anchor nodes to the top, bottom, left side, or center of the layout.

The GridPane class enables the developer to create a flexible grid of rows and columns in which to lay out content nodes.

To achieve a desired layout structure, different containers can be nested within a JavaFX application.

Posizionamento assoluto: Pane

```
public void start(Stage primaryStage) {
    primaryStage.setTitle("Hello World!");
    Button btn = new Button();
    btn.setText("'Hello World'");
    btn.setLayoutX(250);
    btn.setLayoutY(220);
    Pane pane= new Pane();
    pane.getChildren().add(btn);
    Group root = new Group(pane);
    primaryStage.setScene(new Scene(root, 300, 250));
    primaryStage.show();
}
```

Diciamo alla componente dove vogliamo che lei si posizioni

In generale, *da evitare*!

Il posizionamento di una componente è stabilito dalla somma di *LayoutX* e *TranslateX*. Però *LayoutX* è definito dal contenitore, e in genere è statico, quindi cambiarlo a mano non serve a nulla, però si può lavorare su *TranslateX*

Dalle API...

The node's final translation will be computed as layoutX + translateX, where layoutX establishes the node's stable position and translateX optionally makes dynamic adjustments to that position.

If the node is managed and has a Region (or subclass) as its parent, then the layout region will set layoutX according to its own layout policy.

If the node is unmanaged or parented by a Group, then the application may set layoutX directly to position it.

e finalmente con setTranslateX...

```
class Controller implements EventHandler<ActionEvent>{
      int inc = 0; Circle c; Controller(Circle c)
                        @Override
        public void handle(ActionEvent event) {
                          inc += 10;
                  //c.setLayoutX(inc % 100);
                  //c.setCenterX(inc % 100);
                  c.setTranslateX(inc % 100);
              System.out.println(c.getLayoutX()
       + " " + c.getTranslateX()+" "+c.getCenterX());}}
```



Funziona!

```
-190.0 10.0 200.0
-190.0 20.0 200.0
-190.0 30.0 200.0
-190.0 40.0 200.0
```

Dimensionamento e allineamento

È possible dimensionare direttamente gli elementi

- Es., `btn.setPrefWidth(200);`

... oppure specificare vincoli sulle dimensioni

- Es., `btn.setMinWidth(100);`

Per esempi di dimensionamento e allineamento:
https://docs.oracle.com/javase/8/javafx/layout-tutorial/size_align.htm

L'idea è di strutturare tutti gli elementi che compongono l'interfaccia grafica in un albero

Application

Ogni classe che vuole usare l’interfaccia grafica di Java FX deve estendere la classe *Application*.
NON BISOGNA MAI creare un costruttore non vuoto di una classe che estende Application
NON BISOGNA MAI istanziare una classe che estende Application.
Questo perché è il framework che chiama il costruttore e fa tutto il resto.
Volendo si possono mettere le istanziazioni delle variabili di istanza o nel metodo start, oppure in un costruttore vuoto (anche se è meglio non toccare un costruttore) oppure direttamente nella dichiarazione delle variabili di istanza.